

# Java day wise

## Day 2

---

### Garbage Collection (GC) in Java

#### What is Garbage Collection?

Garbage Collection (GC) is a process in Java used to automatically identify and remove objects from **Heap memory** that are **no longer in use**. This helps prevent **memory leaks** and ensures efficient memory management.

#### Key Concept

- **Garbage = Un-referencable object** (An object that no longer has any valid references).
  - GC **freeds up memory** by deleting these objects, allowing new object allocations.
- 

#### Why Automatic Garbage Collection?

Java provides **Automatic Garbage Collection** to:

- ✓ Prevent **memory leaks**
  - ✓ Avoid **manual memory management** (unlike C/C++)
  - ✓ Improve **application performance**
  - ✓ Ensure **better memory optimization**
- 

#### How Garbage Collection Works in Java?

The **Java Virtual Machine (JVM)** manages memory using **Garbage Collection**, which runs as a **daemon thread** (background process).

#### JVM Creates Two Threads:

- 1 **Main Thread** → Executes `main()` method **sequentially** (Foreground thread).
- 2 **Garbage Collector (GC) Thread** → A **daemon thread** that runs in the **background**, activated **only when needed**.

GC automatically **releases memory** occupied by **unreferenced objects** allocated on the **heap**.

---

#### How to Request Garbage Collection?

Though Java has **automatic GC**, you can request it manually. However, **JVM may ignore the request**.

- 1 **Using System Class API**

```
System.gc(); // Requests garbage collection
```

## 2 Using Runtime Class API

```
Runtime.getRuntime().gc(); // Alternative way to request GC
```

## 3 Using `finalize()` Method (Deprecated in Java 9+)

The `finalize()` method is automatically called by GC **before deleting an object**.

```
@Override
protected void finalize() throws Throwable {
    System.out.println("Object is getting garbage collected");
}
```

### ▲ Note:

- `finalize()` is **deprecated in Java 9** because it causes **unpredictable behavior** and **performance issues**.
- Instead, use **try-with-resources** or explicit cleanup methods.

## What Garbage Collection Does Not Handle?

Garbage Collector **does NOT** handle **non-Java resources**, such as:

✗ **File handles**

✗ **Database connections**

✗ **Network sockets**

**Solution:** You must explicitly close these resources using:

```
try (FileInputStream fis = new FileInputStream("file.txt")) {
    // Use the file
} catch (IOException e) {
    e.printStackTrace();
}
// File automatically closed due to try-with-resources
```

## When is an Object Eligible for Garbage Collection?

An object becomes a **candidate for GC** when there are **no valid references** pointing to it.

### Scenarios where an object is marked for GC:

#### 1 Nullifying a Reference

```
Box b1 = new Box(1,2,3);
```

```
b1 = null; // The Box object is now eligible for GC
```

## 2 Reassigning a Reference

```
Box b1 = new Box(10,20,30);  
b1 = new Box(11,21,31); // First Box object is now unreferenced (eligible for GC)
```

## 3 Object Created Inside a Method (Local Scope)

- If an object is **created inside a method** and **its reference is not returned**, it becomes unreachable after method execution.

```
void createBox() {  
    Box b1 = new Box(5, 10, 15);  
    // b1 exists only inside this method  
} // Once method execution ends, b1 is eligible for GC
```

## 4 Island of Isolation

- If **two objects refer to each other** but have **no external references**, they form an **"island of isolation"** and are **eligible for GC**.

```
class Test {  
    Test t;  
}  
  
Test t1 = new Test();  
Test t2 = new Test();  
t1.t = t2;  
t2.t = t1;  
  
t1 = t2 = null; // Both objects are now unreachable (eligible for GC)
```

# Types of Garbage Collectors in Java

Java provides different types of **Garbage Collectors**, each optimized for different workloads.

## 1 Serial GC

- **Single-threaded**
- Best for **small applications**

## 2 Parallel GC (Throughput GC)

- Uses **multiple threads**
- Good for **multi-core processors**

## 3 G1 (Garbage-First) GC (Default in Java 9+)

- Divides heap into **regions**
- **Low pause time**, suitable for large applications

#### 4 ZGC & Shenandoah GC

- **Low-latency collectors**
  - Handles **large heaps (>100GB)** efficiently
- 

## Final Thoughts

- ✓ **Garbage Collection** helps Java applications manage memory automatically.
  - ✓ JVM **removes objects** that are **no longer referenced**.
  - ✓ You can **request GC**, but **JVM decides when to execute it**.
  - ✓ **Some resources (files, DB connections)** must be **closed manually**.
  - ✓ **Different GC algorithms** optimize performance based on application needs.
- 

This version is **more structured, clear, and includes practical examples**. 🚀

Let me know if you need further modifications or explanations! 😊

## Statically Typed vs Dynamically Typed Languages

### Statically Typed Languages:

- Variable types are **known at compile time**.
- Once declared, a variable **cannot change its type**.
- Examples: **C, C++, Java**

### Dynamically Typed Languages:

- Variable types are **determined at runtime**.
  - A variable **can hold different types of values over time**.
  - Examples: **Python, Ruby**
- 

## Java: A Statically & Strongly Typed Language

- **Each data type** (int, boolean, char, etc.) is **predefined** in the Java Language Specification (JLS).
  - **Every variable** must be explicitly declared with a **specific data type**.
- 

## Java Data Types

### 1 Primitive Data Types (Hold values directly)

- **Boolean**: `true` or `false` (Size: JVM-specific, Default: `false` )
- **Byte**: 8-bit integer (-128 to 127, Default: `0` )
- **Short**: 16-bit integer (-32,768 to 32,767, Default: `0` )

- **Int:** 32-bit integer ( $-2^{31}$  to  $2^{31}-1$ , Default: `0` )
- **Long:** 64-bit integer ( $-2^{63}$  to  $2^{63}-1$ , Default: `0L` )
- **Float:** 32-bit floating-point (~7 decimal places, Default: `0.0f` )
- **Double:** 64-bit floating-point (~16 decimal places, Default: `0.0` )
- **Char:** 16-bit Unicode character ( `'\u0000'` to `'\uffff'` , Default: `'\u0000'` )

 **Note:** Java uses **Unicode** (not ASCII), which is why `char` takes **2 bytes**.

---

## 2 Non-Primitive (Reference) Data Types (Hold memory addresses)

- **Class Type:** Objects of user-defined classes.
- **Array Type:** Collections of similar data types.
- **Interface Type:** Defines methods without implementation (e.g., `Comparator` ).

### Objects

- Objects are instances of a class with **state** (attributes), **behavior** (methods), and **identity**.

### Arrays

- Dynamically allocated, indexed from `0` , and have a `length` property.

### Interfaces

- Abstract methods without implementation, used for defining behavior (e.g., `Comparator` ).
- 

### Summary

- **Java is statically & strongly typed:** Every variable has a **fixed** data type.
- **Primitive types** store values, while **reference types** store memory addresses.
- **Objects, arrays, and interfaces** are non-primitive types.

## Object-Oriented Programming (OOP)

OOP is a programming paradigm that **models real-world entities** using objects and classes. It focuses on **data (state)** and **behavior (methods)** rather than just functions.

### Key Principles of OOP:

1. **Encapsulation** – Bundling data and methods together and restricting direct access.
  2. **Abstraction** – Hiding internal implementation and exposing only the necessary details.
  3. **Inheritance** – Allowing a class to acquire properties and methods of another class.
  4. **Polymorphism** – Ability to take multiple forms (method overloading & overriding).
- 

## Class & Object

### Class

A **class** is a **blueprint** that defines properties (**fields/variables**) and behaviors (**methods**) of objects.

**Example:**

```
class Car {
    String brand;
    int speed;

    Car(String brand, int speed) { // Constructor
        this.brand = brand;
        this.speed = speed;
    }

    void display() {
        System.out.println(brand + " is running at " + speed + " km/h");
    }
}
```

## Object

An **object** is an instance of a class, created using the `new` keyword.

**Example:**

```
Car myCar = new Car("Tesla", 120);
myCar.display(); // Output: Tesla is running at 120 km/h
```

## Encapsulation

Encapsulation is **data hiding** and **controlled access** through methods.

- **Achieved using:**
  - ✓ **Private** variables to restrict direct access.
  - ✓ **Public** getter & setter methods for controlled access.

**Example:**

```
class Person {
    private String name;

    public void setName(String name) { // Setter
        this.name = name;
    }

    public String getName() { // Getter
        return name;
    }
}
```

## Advantages of Encapsulation

- ✓ **Data Security** – Prevents unauthorized access.
- ✓ **Flexibility** – Control how data is accessed and modified.
- ✓ **Maintainability** – Easy to update without affecting other parts of the program.

## Summary

- OOP models real-world entities using classes & objects.
- Encapsulation secures data by restricting direct access.
- Encapsulation = Data Hiding + Controlled Access (Getters & Setters).

## Day 3

### Java Arrays

#### What is an Array?

An **array** is a **fixed-size** data structure that stores multiple values of the **same data type** in contiguous memory locations.

✓ **Example:** If we need to store names of 100 people, we use an array instead of 100 separate variables.

```
String[] names = new String[100]; // Can store exactly 100 names
```

#### Characteristics of Arrays in Java

- **Stored on the heap memory.**
- **Fixed size** (cannot be changed after declaration).
- **Can hold both primitive & reference data types.**
- **Zero-based indexing** (first element at index `0`).
- **Automatically initialized with default values.**

## Declaring and Initializing Arrays

### 1. Declaration (Only reference created, no memory allocated)

```
int[] numbers; // Declaring an array reference
```

### 2. Allocation (Memory is allocated using `new` )

```
numbers = new int[5]; // Memory allocated for 5 integers
```

OR

```
int[] numbers = new int[5]; // Declaration & allocation together
```

### 3. Initialization (Assigning values to elements)

```
int[] numbers = {10, 20, 30, 40, 50}; // Direct initialization
```

### 4. Accessing Elements

```
System.out.println(numbers[0]); // Output: 10
```

### 5. Finding Array Length

```
System.out.println(numbers.length); // Output: 5
```

## Multidimensional Arrays

Arrays within arrays (e.g., matrix representation).

#### ✓ Example: 2D Array (Matrix of 3 rows & 4 columns)

```
int[][] matrix = new int[3][4]; // 3×4 matrix
```

#### ✓ Example: Initializing a 2D array

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

#### ✓ Accessing Elements in 2D Array

```
System.out.println(matrix[1][2]); // Output: 6
```

## Summary

- **Arrays** store multiple values of **same type** in a contiguous manner.
- **Size is fixed** once declared.
- **Elements accessed via index** ( **0** to **length-1** ).
- **Can be single-dimensional or multi-dimensional (e.g., 2D array).**

## Packages in Java

### What is a Package?

A **package** is a collection of **related classes and interfaces** that are grouped together for better organization and modularity.

It helps in **avoiding name conflicts** and provides **better access control**.

## Why Use Packages?

- ✓ **Organizes classes** based on functionality.
- ✓ **Avoids name conflicts** (same class names in different packages).
- ✓ **Encapsulation & access control** (controlled visibility).
- ✓ **Code reusability** across projects.

## Types of Packages in Java

1. **Built-in Packages (Predefined)** – Provided by Java (e.g., `java.util`, `java.io`).
2. **User-defined Packages** – Created by developers for modular programming.

## Creating a User-defined Package

### Steps to Create a Package

1. **Define the package** at the beginning of the Java file.
2. **Compile the file** to create the package structure.
3. **Use the package** in another class.

### Example: Creating and Using a Package

#### Step 1: Create a Package

Save this file as `MyPackageClass.java` inside a `mypackage` folder.

```
// Package declaration (must be the first statement)
package mypackage;

public class MyPackageClass {
    public void display() {
        System.out.println("Hello from MyPackageClass!");
    }
}
```

#### Step 2: Compile the Java File

Go to the folder where the file is stored and run:

```
javac -d . MyPackageClass.java
```

This will create a `mypackage` folder and store the compiled class inside it.

#### Step 3: Use the Package in Another Class

Create another file `Main.java` outside `mypackage` folder.

```
// Importing the package
import mypackage.MyPackageClass;

public class Main {
    public static void main(String[] args) {
        MyPackageClass obj = new MyPackageClass();
        obj.display();
    }
}
```

## Step 4: Compile and Run

```
javac Main.java
java Main
```

Output:

```
Hello from MyPackageClass!
```

## How to Set the Classpath for Packages?

If you want to run the Java program from **any directory**, you need to set the **classpath**.

### Setting Classpath Temporarily (Command Line)

```
set classpath=C:\path\to\your\bin;
```

### Setting Classpath Permanently (Windows)

1. Right-click **This PC** → Properties → Advanced system settings → Environment Variables.
2. Find **CLASSPATH** (or create a new one).
3. Set value as `C:\path\to\your\bin;`
4. Click **OK** → Restart the terminal.

## Rules for Using Packages

1. The `package` statement must be the first line in the source file (before imports).
2. Import statements must be written **after the package declaration** but **before class definitions**.
3. **All classes in a source file belong to the same package** (you cannot mix packages in a single `.java` file).

## Day 4

That was a **comprehensive** overview of **Inheritance & Polymorphism** in Java! Let me break it down into a **structured explanation** with clear **examples** to solidify the concepts.

---

## Inheritance in Java

Inheritance is the process where a **child class (subclass)** acquires properties and behavior from a **parent class (superclass)**.

### ✅ Why Use Inheritance?

- Promotes **code reusability** (common code in a superclass).
- **Avoids redundancy**, making code more maintainable.
- Supports **hierarchical classification** (general → specialized).
- Allows **method overriding** for runtime polymorphism.

### ✅ Represents an IS-A Relationship

Example:

- **Car IS-A Vehicle**
- **Dog IS-A Animal**
- **Manager IS-A Employee**

---

## Types of Inheritance

### 1. Single Inheritance

One class inherits from another.

```
class Animal {
    void eat() { System.out.println("This animal eats food."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Dog barks!"); }
}

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // Inherited from Animal
        d.bark(); // Defined in Dog
    }
}
```

### ✅ Output:

```
This animal eats food.
```

Dog barks!

## 2. Multilevel Inheritance

A class inherits from a derived class.

```
class Animal {
    void eat() { System.out.println("This animal eats food."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Dog barks!"); }
}

class Puppy extends Dog {
    void weep() { System.out.println("Puppy weeps!"); }
}

public class Test {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat(); // Inherited from Animal
        p.bark(); // Inherited from Dog
        p.weep(); // Defined in Puppy
    }
}
```

### ✓ Output:

This animal eats food.  
Dog barks!  
Puppy weeps!

## 3. Hierarchical Inheritance

Multiple child classes inherit from a single parent.

```
class Animal {
    void eat() { System.out.println("This animal eats food."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Dog barks!"); }
}

class Cat extends Animal {
    void meow() { System.out.println("Cat meows!"); }
}
```

```

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();

        c.eat();
        c.meow();
    }
}

```

✅ **Output:**

```

This animal eats food.
Dog barks!
This animal eats food.
Cat meows!

```

## 4. Multiple Inheritance (NOT Supported in Java with Classes)

Java does **NOT** support **multiple inheritance with classes** because of the **diamond problem**.

✅ **Solution?** Java supports multiple inheritance using **interfaces**.

```

interface A {
    void show();
}

interface B {
    void display();
}

class C implements A, B {
    public void show() { System.out.println("Hello from A"); }
    public void display() { System.out.println("Hello from B"); }
}

public class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.show();
        obj.display();
    }
}

```

### ✅ Output:

```
Hello from A
Hello from B
```

## Method Overriding (Runtime Polymorphism)

When a **child class** provides its **own implementation** of a method present in the **parent class**.

### ✅ Rules for Overriding:

1. **Method name and signature** must be the same.
2. **Return type must be the same** (or a subclass - covariance).
3. **Access modifier cannot be more restrictive** than the superclass method.
4. **Cannot override final or static methods.**

### Example: Overriding a Method

```
class Parent {
    void display() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child class method");
    }
}

public class Test {
    public static void main(String[] args) {
        Parent p = new Child(); // Upcasting
        p.display(); // Calls overridden method in Child class
    }
}
```

### ✅ Output:

```
Child class method
```

👉 **JVM decides at runtime to call the overridden method in Child.**

## Method Overloading (Compile-time Polymorphism)

### ✅ Rules for Overloading:

- **Same method name, but different signatures** (different parameters).
- **Return type does not matter.**

### Example: Method Overloading

```
class MathUtil {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}

public class Test {
    public static void main(String[] args) {
        MathUtil obj = new MathUtil();
        System.out.println(obj.add(5, 10));    // Calls int version
        System.out.println(obj.add(5.5, 10.5)); // Calls double version
    }
}
```

#### ✓ Output:

```
15
16.0
```

### Key Differences: Overloading vs Overriding

| Feature                | Overloading                            | Overriding                                              |
|------------------------|----------------------------------------|---------------------------------------------------------|
| <b>Definition</b>      | Same method name, different parameters | Same method name & parameters, different implementation |
| <b>Binding Time</b>    | Compile-time (static)                  | Runtime (dynamic)                                       |
| <b>Return Type</b>     | Can be different                       | Must be same (or subclass - covariance)                 |
| <b>Access Modifier</b> | No restrictions                        | Cannot reduce visibility                                |
| <b>Usage</b>           | Improves code readability              | Achieves runtime polymorphism                           |

### super & this Keyword

| Keyword      | Purpose                                  |
|--------------|------------------------------------------|
| <b>this</b>  | Refers to the <b>current object</b>      |
| <b>super</b> | Refers to the <b>parent class object</b> |

### Using **super** for Method Access

```
class Parent {
    void display() {
        System.out.println("Parent method");
    }
}
```

```

    }
}

class Child extends Parent {
    void display() {
        super.display(); // Calls Parent class method
        System.out.println("Child method");
    }
}

public class Test {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display();
    }
}

```

#### ✓ Output:

```

Parent method
Child method

```

## Final Notes

- **Polymorphism** allows the same method to behave differently based on the object.
- **Overloading** is resolved at compile-time, while **overriding** is resolved at runtime.
- **super()** calls the parent class constructor, and **this()** calls the constructor in the same class.
- Java **does not support multiple inheritance** to avoid ambiguity (diamond problem).

That was a **detailed** breakdown of **Inheritance & Polymorphism** in Java. Do you want to practice some **coding exercises** to test your understanding? 😊

## Understanding **static** in Java

The **static** keyword in Java allows you to define **class-level members** rather than **instance-level members**. Let's go over its main usages:

### 1. Static Data Members (Class Variables)

- Stored **only once** in **Metaspace** or a **special heap area**.
- Shared **across all instances** of the class.
- Default values: **0**, **null**, **false**, etc.

#### Example:

```

class Emp {
    public static int idCounter; // Shared across all instances
}

```

```
private String name;
private int empld;
}
```

If **10 Emp objects** are created:

- **idCounter** : **1 copy** (shared across objects)
- **name** : **10 copies** (separate for each object)
- **empld** : **10 copies** (separate for each object)

## 2. Static Methods

- Can be called **without creating an instance** ( `ClassName.method()` ).
- **Cannot use** `this` or `super`, as they are related to instances.

### Rules:

✅ Can static methods access static members? → Yes

❌ Can static methods access non-static members? → No

✅ Can non-static methods access static members? → Yes

### Example:

```
class A {
    private int i; // Non-static
    private static int j; // Static

    public static void show() {
        // System.out.println(i); // ❌ Error! Can't access non-static member
        System.out.println(j); // ✅ Allowed
    }
}
```

## 3. Static Import

- Allows you to access **static members directly** without class name.

### Example:

```
import static java.lang.System.out;
import static java.lang.Math.*;

public class StaticImportExample {
    public static void main(String[] args) {
        out.println("Static Import Example!"); // Instead of System.out.println()
        out.println(sqrt(16)); // Instead of Math.sqrt(16)
        out.println(PI); // Instead of Math.PI
    }
}
```

```
}  
}
```

## 4. Static Initializer Block

- Runs **once** when the class is **loaded** into memory.
- Used for **static variable initialization** or **framework setup (Spring, Hibernate, etc.)**.

### Example:

```
class StaticBlockExample {  
    static int data;  
  
    // Static initializer block  
    static {  
        System.out.println("Static block executed!");  
        data = 100;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Value of data: " + data);  
    }  
}
```

✓ Runs **only once** when the class is loaded.

## 5. Non-Static Initializer Block (Instance Block)

- Runs **before any constructor call** during object creation.
- **Better alternative** → Use parameterized constructors.

### Example:

```
class InstanceInitializerExample {  
    int x;  
  
    // Instance initializer block  
    {  
        System.out.println("Instance block executed!");  
        x = 50;  
    }  
  
    public InstanceInitializerExample() {  
        System.out.println("Constructor executed!");  
    }  
  
    public static void main(String[] args) {  
        new InstanceInitializerExample();  
    }  
}
```

```
}  
}
```

✓ Runs **once per object creation** (before constructor).

## 6. Static Nested Classes

- A **static class inside another class**.
- Can **only access static members** of the outer class **directly**.

### Example:

```
class Outer {  
    static int x = 10;  
  
    static class Nested {  
        void display() {  
            System.out.println("Value of x: " + x); // ✓ Can access static members  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Outer.Nested nested = new Outer.Nested();  
        nested.display();  
    }  
}
```

✓ Used for **better organization** and **encapsulation**.

## Conclusion

- `static` helps manage **class-level** members efficiently.
- It **reduces memory usage** by avoiding **unnecessary duplication** of data.
- **Best practices:** Use `static` **wisely** for **constants, utility methods, configuration setup, and shared counters**.

## 1. Inheritance & Its Types

- Inheritance follows **IS-A** relationship for reusability.
- **Types of inheritance:**
  - **Single** ( `class B extends A` )
  - **Multi-level** ( `class C extends B extends A` )
  - **Hierarchical** ( `class B, C, D extends A` )

- **Multiple inheritance NOT supported** due to the **diamond problem**.

---

## 2. Polymorphism & Method Binding

- **Compile-time polymorphism (Method Overloading)** → Decided by **Java Compiler**.
  - Overloaded methods must have **different parameters** (not just return type).
- **Run-time polymorphism (Method Overriding)** → Decided by **JVM**.
  - JVM resolves method binding based on **object type** (not reference type).
  - `@Override` annotation ensures proper method overriding.

---

## 3. `super` and `this` Keywords

- `super(...)` → Calls **parent class constructor**.
- `this(...)` → Calls **same class constructor**.
- **Rules:**
  - Only **constructors** can use `super(...)` or `this(...)`.
  - It must be the **first statement** inside a constructor.
  - A constructor **can't use both** `super(...)` and `this(...)` together.

---

## 4. `Object` Class & Overriding `toString()` & `equals()`

- `Object` is the **universal superclass** of all Java classes.
- Default `toString()` returns → **ClassName@HashCode**.
- **Override** `toString()` → To display **actual object details**.
- **Override** `equals()` → To compare **content, not references**.

---

## 5. `instanceof` for Run-time Type Checking

- Used to **check an object's type** at runtime.
- Example:

```
if (obj instanceof Student) {  
    Student s = (Student) obj; // Downcasting after checking  
}
```

---

## 6. `protected` Keyword Behavior

- **Within the same package** → Works like `default` (accessible to all).
- **Outside the package** → Accessible **only via inheritance**, not through object references.

---

## Understanding `instanceof` in Java

The `instanceof` operator is used for **run-time type checking (RTTI)** to determine if an object belongs to a particular class, subclass, or implements an interface.

## Key Points

1. **Works at Runtime** → Checks the actual object type, **not** the reference type.
2. **Supports Inheritance** → A subclass object is also an instance of its superclass.
3. **Checks Interfaces** → Can verify if an object implements an interface.
4. **Safe Casting** → Prevents `ClassCastException` by confirming the type before casting.
5. **Handles `null` Gracefully** → Returns **false** if the object is `null`.

## Example Explanation

```
class Emp {}
class Mgr extends Emp {}
class SalesMgr extends Mgr {}
class Worker extends Emp {}

public class InstanceofDemo {
    public static void main(String[] args) {
        Emp e = new Mgr(); // Upcasting (Superclass ref to Subclass object)

        System.out.println(e instanceof Mgr);    // true  (e is actually a Mgr)
        System.out.println(e instanceof Emp);    // true  (Mgr is a subclass of Emp)
        System.out.println(e instanceof Object); // true  (All classes extend Object)
        System.out.println(e instanceof SalesMgr); // false (e is Mgr, not SalesMgr)
        System.out.println(e instanceof Worker); // false (Mgr and Worker are unrelated)

        e = null;
        System.out.println(e instanceof Emp);    // false (null is not an instance of any class)
    }
}
```

## Output:

```
true
true
true
false
false
false
```

## Breakdown of Output

### 1. `e instanceof Mgr` → `true`

- `e` refers to a `Mgr` object.
- Since `Mgr` is being checked, **it matches exactly**.

2. `e instanceof Emp` → `true`

- `Mgr` is a subclass of `Emp`, meaning all `Mgr` objects are also `Emp`.

3. `e instanceof Object` → `true`

- All Java objects implicitly inherit from `Object`, so `Mgr` is an instance of `Object`.

4. `e instanceof SalesMgr` → `false`

- `e` is of type `Mgr`, not `SalesMgr`.
- Even though `SalesMgr` is a subclass of `Mgr`, `e` was never instantiated as `SalesMgr`.

5. `e instanceof Worker` → `false`

- `Worker` is not related to `Mgr` (both extend `Emp` but are different subclasses).

6. `e = null; e instanceof Emp` → `false`

- `instanceof` always returns `false` for `null` values.

## When to Use `instanceof` ?

### ✓ Before Casting:

```
if (e instanceof Mgr) {  
    Mgr m = (Mgr) e; // Safe casting  
}
```

### ✓ For Type-Specific Operations:

```
void printEmployeeType(Emp e) {  
    if (e instanceof Mgr) {  
        System.out.println("Manager");  
    } else if (e instanceof Worker) {  
        System.out.println("Worker");  
    } else {  
        System.out.println("Other Employee");  
    }  
}
```

## Best Practices

✗ **Avoid `instanceof` for Polymorphism:** Instead, use method overriding for dynamic behavior.

✓ **Use `instanceof` for Safe Casting:** Helps avoid `ClassCastException`.

✓ **Check for `null` First:** Prevents unnecessary checks.

## Do Subclasses Inherit Private Data Members from the Superclass?

Answer: NO!

Explanation:

1. **Private members ( `private` fields and methods) are NOT accessible in the subclass.**

- They are **not inherited** in the sense that they cannot be directly accessed in the subclass.
- However, they **are still present** in memory inside the subclass object.

2. **What does this mean?**

- When a subclass object is created, the **private members from the superclass exist within it**.
- But they **can only be accessed using public or protected getter/setter methods** from the superclass.

## Example to Understand

```
class Person {
    private String firstName; // Private member
    private String lastName; // Private member

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    // Public method to access private data
    public String getFullName() {
        return firstName + " " + lastName;
    }
}

class Student extends Person {
    private int gradYear;
    private String course;
    private double fees;
    private int marks;

    public Student(String firstName, String lastName, int gradYear, String course, double fees, int marks) {
        super(firstName, lastName); // Calling the superclass constructor
        this.gradYear = gradYear;
        this.course = course;
        this.fees = fees;
        this.marks = marks;
    }

    // Can't access firstName or lastName directly!
    public void printDetails() {
        // System.out.println("Student Name: " + firstName + " " + lastName); // ❌ ERROR
        System.out.println("Student Name: " + getFullName()); // ✅ Correct way
    }
}
```

```

}

public class TestInheritance {
    public static void main(String[] args) {
        Student s = new Student("Akshay", "Kumar", 2025, "PG-DAC", 50000, 90);
        s.printDetails();
    }
}

```

## Key Observations from the Example

1. `firstName` and `lastName` exist in the `Student` object (stored in heap memory).
2. However, they **cannot be accessed directly** inside `Student`.
3. The correct way to access them is through **public methods like** `getFullName()`.

## Memory Representation

A `Student` object contains **all attributes from** `Person` **plus its own**, so it has:

**CP + 6 slots**

- **CP (Common Parent)** → `Object` class fields
- **2 slots from** `Person` ( `firstName` , `lastName` )
- **4 slots from** `Student` ( `gradYear` , `course` , `fees` , `marks` )

So in memory, a `Student` object will have **6 slots in total** for its attributes.

## Conclusion

- ✓ **Private members exist in subclass objects but are NOT accessible directly.**
- ✓ **Use `public` or `protected` getters/setters to access private fields from the superclass.**
- ✓ **Each subclass instance still contains all inherited fields in memory.**

## Understanding Upcasting in Java

### Definition

Upcasting is the process of converting a **subclass type reference** into a **superclass type reference**.

It is an **implicit** (automatic) type conversion in Java.

### Key Concept: "IS-A" Relationship

- A **subclass object can be referenced by a superclass variable** without explicit casting.
- **Every** `Student` is a `Person` , so we can say:

```
Person p = new Student(...); // ✓ Upcasting (Implicit)
```

## Example 1: Upcasting

```

class Person {
    void display() {
        System.out.println("I am a Person");
    }
}

class Student extends Person {
    void show() {
        System.out.println("I am a Student");
    }
}

public class UpcastingExample {
    public static void main(String[] args) {
        Person p = new Student(); // Upcasting
        p.display(); // ✅ Allowed (Inherited method)

        // p.show(); // ❌ ERROR: Cannot call subclass-specific methods with superclass reference
    }
}

```

#### 📌 Why is `p.show();` not allowed?

The `p` reference is of type `Person`, which does **not** have a `show()` method.

Although `p` is referring to a `Student` object, **Java allows access only to the members of `Person`**.

## Example 2: Fruit and Mango

```

class Fruit {
    void taste() {
        System.out.println("No Specific Taste");
    }
}

class Mango extends Fruit {
    void taste() {
        System.out.println("Mango is Sweet");
    }
}

public class TestUpcasting {
    public static void main(String[] args) {
        Fruit f = new Mango(); // ✅ Upcasting
        f.taste(); // 🔥 Output: "Mango is Sweet" (Dynamic method dispatch)
    }
}

```

---

## Dynamic Method Dispatch (Runtime Polymorphism)

- When we **call an overridden method on an upcasted object**, Java resolves it **at runtime**.
- This behavior is called **dynamic method dispatch**.

### ◆ Example:

```
Fruit f = new Mango();  
f.taste(); // ✅ Calls Mango's `taste()` method, NOT Fruit's method
```

### 👉 Why?

At runtime, the JVM looks at the **actual object type** ( **Mango** ) instead of the reference type ( **Fruit** ), and executes the overridden **taste()** method.

---

## Key Takeaways

- ✓ **Upcasting is implicit (automatic) and safe.**
- ✓ **Upcasting allows only superclass methods to be accessed.**
- ✓ **If a subclass overrides a method, the subclass version is executed (Dynamic Method Dispatch).**
- ✓ **To access subclass-specific methods, we need downcasting ( **(Student) p** ).**

## Day 6

### Abstraction in Java

#### Definition

**Abstraction** is the concept of hiding **implementation details** while exposing only the essential features to the user.

#### ◆ Example:

An **ATM** allows users to withdraw or deposit money but hides the **internal mechanism** of how it interacts with the bank.

#### ◆ Key Idea:

- Focus on **what** an object does, rather than **how** it does it.
- Achieved using **abstract classes** and **interfaces**.

---

## How is Abstraction Achieved in Java?

### 1 Using Abstract Classes

An **abstract class** is a class that **cannot be instantiated** and may contain **abstract methods** (methods without implementation).

#### Abstract Class Syntax:

```

abstract class BoundedShape {
    int x, y;

    abstract double area();

    void display() {
        System.out.println("Bounded Shape Display");
    }
}

```

## Rules for Abstract Classes

- ✓ Can have **both abstract and concrete methods**.
- ✓ Cannot be **instantiated** directly.
- ✓ Can have **constructors** to initialize data members.
- ✓ Must be **extended** by a subclass to provide implementations.

## Example: Abstract Class in Action

```

abstract class Employee {
    int empld;
    String dept;

    Employee(int empld, String dept) {
        this.empld = empld;
        this.dept = dept;
    }

    abstract void work();
}

class Manager extends Employee {
    Manager(int empld, String dept) {
        super(empld, dept);
    }

    void work() {
        System.out.println("Managing team");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Manager(101, "HR");
        e.work();
    }
}

```

```
}  
}
```

## 2 Using Interfaces

An **interface** provides **100% abstraction** by defining only method **signatures** (until Java 8).

### Interface Syntax

```
interface Computable {  
    int DATA = 100;  
    double compute(double d1, double d2);  
}
```

### Example: Interface in Action

```
interface Drawable {  
    void draw();  
}  
  
class Circle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Drawable d = new Circle();  
        d.draw();  
    }  
}
```

#### ◆ Interfaces vs. Abstract Classes:

| Feature     | Abstract Class                       | Interface                                 |
|-------------|--------------------------------------|-------------------------------------------|
| Abstraction | Partial (0-100%)                     | Full (100%)                               |
| Methods     | Can have abstract & concrete methods | Only abstract methods (Java 7)            |
| Fields      | Can have instance variables          | Only <b>public static final</b> constants |
| Constructor | Can have constructors                | Cannot have constructors                  |

### Advanced Abstraction Concepts

Can a Class be Both **abstract** and **final** ?

🚫 No!

- **abstract** → Requires subclasses to implement methods.

- `final` → Prevents subclassing.
- These two contradict each other.

## Final Keyword in Java

### 1. Final Variables (Immutable Constants)

```
public class Test {  
    final int MAX = 100;  
    void show() {  
        // MAX = 200; // Compilation error  
    }  
}
```

### 1. Final Methods (Cannot be Overridden)

```
class A {  
    final void display() {  
        System.out.println("Final Method");  
    }  
}
```

### 1. Final Class (Cannot be Extended)

```
final class Base {}
```

### 1. Final Reference Variables (Reference cannot be changed)

```
final Employee e = new Manager(101, "HR");
```

## Interface in Java - Detailed Explanation

### 1 Multiple Inheritance via Interfaces

```
interface Printable {  
    void print();  
}  
  
interface Showable {  
    void show();  
}  
  
class A implements Printable, Showable {  
    public void print() { System.out.println("Hello"); }  
    public void show() { System.out.println("Welcome"); }
```

```

public static void main(String args[]) {
    A obj = new A();
    obj.print();
    obj.show();
}
}

```

## 2 Functional Interface (Single Abstract Method - SAM)

◆ **Definition:** An interface with exactly **one abstract method**.

```

@FunctionalInterface
interface MyFunctionalInterface {
    void display();
}

```

📌 **Examples in Java:**

- `Runnable` (for multithreading)
- `Comparable` (for sorting)

◆ **Lambda Expression Compatibility**

```

@FunctionalInterface
interface Sayable {
    void say();
}

public class LambdaExample {
    public static void main(String[] args) {
        Sayable s = () → System.out.println("Hello World");
        s.say();
    }
}

```

✅ **Lambda expressions work with functional interfaces!**

## 3 Default & Static Methods in Interfaces (Java 8+)

Java 8 introduced **default and static methods** in interfaces.

### Default Method Example

```

interface Vehicle {
    default void start() {
        System.out.println("Vehicle is starting");
    }
}

```

```

class Car implements Vehicle {}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start(); // Output: Vehicle is starting
    }
}

```

## Static Method Example

```

interface MathUtils {
    static int square(int x) {
        return x * x;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtils.square(5));
    }
}

```

## Conclusion

- ✓ **Abstraction** hides unnecessary details, focusing only on essential features.
- ✓ **Abstract classes** provide a blueprint but can also contain some implementation.
- ✓ **Interfaces** allow **100% abstraction** and enable **multiple inheritance**.
- ✓ **Functional interfaces** allow **lambda expressions**.
- ✓ **Java 8** introduced **default and static methods** in interfaces.

Here's a well-structured comparison of **Abstract Classes vs. Interfaces** in Java:

## Abstract Class vs. Interface in Java

Java supports both **abstract classes** and **interfaces** to achieve **abstraction**. While they share some similarities, they have key differences that determine when to use each.

### Key Differences Between Abstract Classes and Interfaces

| Feature                      | Abstract Class                                         | Interface                                                                                           |
|------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Method Type</b>           | Can have <b>both abstract and non-abstract methods</b> | All methods are <b>abstract by default</b> (until Java 8)                                           |
| <b>Multiple Inheritance</b>  | A class <b>can extend only one</b> abstract class      | A class <b>can implement multiple</b> interfaces                                                    |
| <b>Method Implementation</b> | Can have <b>abstract and concrete</b> methods          | Methods are <b>abstract by default</b> , but from Java 8, can have <b>default</b> and <b>static</b> |

|                                     |                                                                                                                       |                                                                                                                          |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
|                                     |                                                                                                                       | methods                                                                                                                  |
| <b>Variable Type</b>                | Can have <b>non-final and final variables</b>                                                                         | All variables are <b>public, static, and final</b> (constants)                                                           |
| <b>Access Modifiers</b>             | Can have <b>private, protected, or public</b> members                                                                 | Members are <b>public by default</b>                                                                                     |
| <b>Constructors</b>                 | Can have <b>constructors</b> (used when instantiated by subclasses)                                                   | Cannot have <b>constructors</b>                                                                                          |
| <b>Extending &amp; Implementing</b> | Can <b>extend another abstract class</b> and <b>implement multiple interfaces</b>                                     | Can <b>extend multiple interfaces</b> , but cannot extend a class                                                        |
| <b>Use Case</b>                     | Best used for <b>hierarchical relationships</b> (e.g., an "Employee" class with subclasses "Manager" and "Developer") | Best used when <b>unrelated classes share common behavior</b> (e.g., "Flyable" interface for both "Bird" and "Airplane") |

## Example: Abstract Class

```

abstract class Animal {
    String name;

    Animal(String name) { // Constructor
        this.name = name;
    }

    abstract void makeSound(); // Abstract method (no body)

    void eat() { // Concrete method
        System.out.println(name + " is eating.");
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    void makeSound() { // Implementing abstract method
        System.out.println(name + " barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog("Buddy");
        myDog.makeSound(); // Output: Buddy barks.
        myDog.eat(); // Output: Buddy is eating.
    }
}

```

- ✓ **Abstract class can have a constructor and concrete methods**
- ✓ **Dog extends Animal and provides an implementation for `makeSound()`**

## Example: Interface

```
interface Flyable {  
    void fly(); // Implicitly public and abstract  
}  
  
class Bird implements Flyable {  
    public void fly() { // Must be public  
        System.out.println("Bird is flying.");  
    }  
}  
  
class Airplane implements Flyable {  
    public void fly() {  
        System.out.println("Airplane is flying.");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Flyable sparrow = new Bird();  
        Flyable boeing = new Airplane();  
  
        sparrow.fly(); // Output: Bird is flying.  
        boeing.fly(); // Output: Airplane is flying.  
    }  
}
```

- ✓ **Multiple classes (Bird & Airplane) implement the same interface**
- ✓ **`Flyable` promotes loose coupling and code reusability**

## When to Use Abstract Classes vs. Interfaces?

| Scenario                                                | Use Abstract Class | Use Interface                           |
|---------------------------------------------------------|--------------------|-----------------------------------------|
| <b>Need partial abstraction (some concrete methods)</b> | ✓ Yes              | ✗ No                                    |
| <b>Need full abstraction (only method signatures)</b>   | ✗ No               | ✓ Yes                                   |
| <b>Need multiple inheritance</b>                        | ✗ No               | ✓ Yes                                   |
| <b>Need constructors</b>                                | ✓ Yes              | ✗ No                                    |
| <b>Need method implementations with common behavior</b> | ✓ Yes              | ✗ No (except default methods in Java 8) |

## Key Takeaways

- ✓ **Use an abstract class** when you have a parent-child relationship (**is-a**).
- ✓ **Use an interface** when multiple unrelated classes should share the same behavior (**can-do**).
- ✓ **Interfaces** support **multiple inheritance**, whereas **abstract classes** do not.
- ✓ **Abstract classes** allow **both abstract and concrete methods**, while **interfaces** (until Java 8) only allowed abstract methods.

Would you like examples with **Java 8+ features like default & static methods?** 🚀

## Day 7

Here's a well-structured breakdown of **Exception Handling in Java** with explanations, syntax, and examples:

---

## Exception Handling in Java

### What is an Exception?

An **exception** is an unwanted or unexpected event that occurs **during runtime**, disrupting the normal flow of a program.

### Common Runtime Errors Leading to Exceptions:

1. **File Not Found** → `FileNotFoundException`
  2. **Accessing Array Out of Bounds** → `ArrayIndexOutOfBoundsException`
  3. **Accessing Methods on a `null` Reference** → `NullPointerException`
  4. **Dividing by Zero** → `ArithmeticException`
- 

### How Exception Handling Works?

1. **JVM detects the exception** and creates a corresponding **exception class instance** (e.g., `NullPointerException`).
  2. **JVM checks if the program provides exception handling code.**
    - **No handling?** JVM aborts the program, prints an error message (Stack Trace).
    - **Handled using `try-catch`?** JVM executes the catch block and continues execution.
- 

## Exception Handling Syntax & Keywords

### Key Java Keywords:

- `try` → Defines the block where exceptions might occur.
- `catch` → Handles the specific exception.
- `finally` → Executes code regardless of whether an exception occurs.
- `throw` → Used to explicitly throw an exception.

- `throws` → Declares exceptions in a method signature.

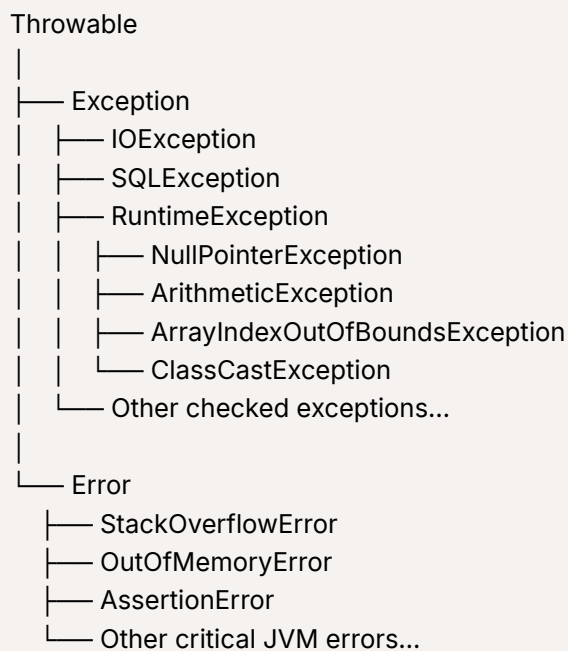
## Basic Example

```
public class ExceptionExample {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0; // ArithmeticException  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero!");  
        }  
        System.out.println("Program continues...");  
    }  
}
```

- ✓ Without `catch`, the program would terminate.
- ✓ With `catch`, execution continues after handling the exception.

## Exception Hierarchy in Java

All exceptions in Java inherit from the `Throwable` class.



## Checked vs. Unchecked Exceptions

### 1 Checked Exceptions (Compile-time)

- Must be handled using `try-catch` OR declared using `throws`.
- Examples:
  - `IOException`

- `SQLException`
- `InterruptedException`

```
import java.io.*;

public class CheckedExceptionExample {
    public static void main(String[] args) throws IOException { // or handle using try-catch
        FileReader file = new FileReader("nonexistent.txt"); // FileNotFoundException
    }
}
```

✅ **Compiler forces us to handle it.**

## 2 Unchecked Exceptions (Runtime)

- Occur due to logic errors; handling is optional.

- **Examples:**

- `NullPointerException`
- `ArithmeticException`
- `ArrayIndexOutOfBoundsException`

```
public class UncheckedExceptionExample {
    public static void main(String[] args) {
        int arr[] = {1, 2, 3};
        System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
    }
}
```

❌ **Compiler does not enforce handling, but it's good practice to do so.**

## Handling Multiple Exceptions

```
try {
    int a = 10 / 0;
    String str = null;
    System.out.println(str.length());
}
catch (ArithmeticException e) {
    System.out.println("Divide by zero error.");
}
catch (NullPointerException e) {
    System.out.println("Null reference error.");
}
catch (Exception e) { // Catch-all block (must be last)
```

```
System.out.println("Some other exception occurred.");  
}
```

✔ Catches multiple exceptions in order, from most specific to most generic.

## Java 7+ Multi-Catch (Pipe | Operator)

```
try {  
    int a = 10 / 0;  
} catch (ArithmeticException | NullPointerException e) {  
    System.out.println("Either divide by zero OR null reference error.");  
}
```

✔ Reduces redundancy when handling multiple exceptions similarly.

## Throws vs. Throw

| Keyword | Purpose                                  | Example                                      |
|---------|------------------------------------------|----------------------------------------------|
| throw   | Used to explicitly throw an exception    | throw new NullPointerException();            |
| throws  | Declares exceptions a method might throw | public void readFile() throws IOException {} |

### Example: Using throw to Throw an Exception

```
public class ThrowExample {  
    static void validateAge(int age) {  
        if (age < 18) {  
            throw new IllegalArgumentException("Age must be 18 or above.");  
        }  
    }  
  
    public static void main(String[] args) {  
        validateAge(16); // Throws IllegalArgumentException  
    }  
}
```

## Finally Block (Always Executes)

```
try {  
    int num = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Error: Divide by zero.");  
}  
finally {
```

```
System.out.println("This block always executes.");
}
```

✓ Executes even if an exception occurs or doesn't.

✗ Only skipped if JVM terminates ( `System.exit(0)` ).

## Try-with-Resources (Java 7+)

- Used for handling resources like files, sockets, etc.
- **Automatically closes resources** after use.

```
import java.io.*;

public class TryWithResourcesExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {
            System.out.println(br.readLine());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

✓ No need to manually close `BufferedReader` . JVM does it automatically.

## Creating Custom Exceptions

1. Extend `Exception` (checked) OR `RuntimeException` (unchecked).
2. Provide constructors to pass messages.

```
class CustomException extends Exception {
    public CustomException(String message) {
        super(message);
    }
}

public class CustomExceptionTest {
    public static void main(String[] args) throws CustomException {
        int balance = 500;
        int withdrawAmount = 1000;

        if (withdrawAmount > balance) {
            throw new CustomException("Insufficient balance.");
        }
    }
}
```

✓ **Best practice: Use custom exceptions for business logic validation.**

## Key Takeaways

- ✓ **Checked exceptions** → Must be handled ( `IOException` ).
- ✓ **Unchecked exceptions** → Result from logic errors ( `NullPointerException` ).
- ✓ **Use `try-catch-finally`** → To handle exceptions.
- ✓ **Use `throw`** → To manually throw exceptions.
- ✓ **Use `throws`** → To declare exceptions in methods.
- ✓ **Try-with-resources** → Auto-closes resources from Java 7+.
- ✓ **Create custom exceptions** → For business rule violations.

Yes! I explained `throw` and `throws` separately with examples, but let's break it down more clearly.

## ◆ `throw` vs. `throws` in Java

| Feature                        | <code>throw</code>                                                    | <code>throws</code>                                       |
|--------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------|
| <b>Purpose</b>                 | Used to explicitly <b>throw an exception</b>                          | Used to <b>declare exceptions</b> a method may throw      |
| <b>Where is it used?</b>       | Inside a method or block                                              | In the method signature                                   |
| <b>Type of exceptions?</b>     | Only one exception at a time                                          | Multiple exceptions (comma-separated)                     |
| <b>Does it stop execution?</b> | Yes, if the exception is not caught                                   | No, just informs the caller to handle it                  |
| <b>Example</b>                 | <code>throw new ArithmeticException("Cannot divide by zero!");</code> | <code>public void readFile() throws IOException {}</code> |

## ◆ Understanding `throw`

- **Used inside a method** to throw an exception **manually**.
- Requires an instance of `Throwable` or its subclass ( `Exception` , `RuntimeException` ).
- **Stops execution** if the exception is not handled.

### ✓ **Example: Using `throw`**

```
public class ThrowExample {
    static void validateAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or above."); // Throwing an exception
        }
    }

    public static void main(String[] args) {
        validateAge(16); // Throws IllegalArgumentException
    }
}
```

```

        System.out.println("This won't execute if an exception occurs.");
    }
}

```

✓ Output:

Exception in thread "main" java.lang.IllegalArgumentException: Age must be 18 or above.

- **Program terminates** because the exception is not caught.

## ◆ Understanding **throws**

- **Used in method signatures** to indicate a method **may throw** exceptions.
- Does **not** handle the exception—just notifies the caller that it must handle it.
- **Mainly used for checked exceptions** ( `IOException` , `SQLException` , etc.).

### ✓ Example: Using **throws**

```

import java.io.*;

public class ThrowsExample {
    // Method declares it may throw an IOException
    public static void readFile() throws IOException {
        FileReader file = new FileReader("nonexistent.txt"); // FileNotFoundException (Checked Exception)
    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) { // Handling exception here
            System.out.println("File not found!");
        }
    }
}

```

✓ Output:

File not found!

- **Compiler enforces handling** of checked exceptions.
- The exception is **handled in** `main()` using `try-catch` .

## ◆ **throw** vs. **throws** in One Example

```

class Bank {
    // Declaring an exception using `throws`
    public void withdraw(int amount) throws Exception {
        if (amount > 1000) {
            throw new Exception("Withdrawal limit exceeded!"); // Throwing an exception
        }
        System.out.println("Withdrawal successful!");
    }
}

public class BankExample {
    public static void main(String[] args) {
        Bank bank = new Bank();
        try {
            bank.withdraw(1500); // Exception thrown here
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

✓ Output:

Error: Withdrawal limit exceeded!

- **throw** is used to generate an exception inside `withdraw()`
- **throws** is used to declare that `withdraw()` may throw an exception
- Handled in `main()` using `try-catch`

## ◆ Summary

| Aspect                                    | throw        | throws                     |
|-------------------------------------------|--------------|----------------------------|
| Used <b>inside</b> methods                | ✓            | ✗                          |
| Used <b>in method signature</b>           | ✗            | ✓                          |
| Stops execution immediately               | ✓            | ✗                          |
| Declares multiple exceptions              | ✗            | ✓                          |
| Requires <code>Throwable</code> instance  | ✓            | ✗                          |
| Must be handled in <code>try-catch</code> | ✗ (optional) | ✓ (for checked exceptions) |

## Day 8

Here's a well-structured and concise summary of **Java 8 Date and Time API**, covering advantages, key classes, methods, and practical examples.

# Java 8 Date and Time API (java.time Package)

Java 8 introduced a new **Date and Time API** under the `java.time` package to overcome the limitations of the **legacy date/time classes** ( `java.util.Date` , `java.util.Calendar` ).

## 1. Limitations of Legacy Date/Time API

1. **Deprecated Constructors** – Many constructors in `java.util.Date` are deprecated.
2. **Complex Formatting** – Requires `SimpleDateFormat` for formatting/parsing.
3. **No Exception for Invalid Dates** – E.g., setting an invalid day/month does not throw an error.
4. **Not Thread-Safe** – `java.util.Date` and `SimpleDateFormat` are mutable and not thread-safe.

## 2. Java 8 New Date/Time API

The new API provides **immutable**, **thread-safe**, and **easier-to-use** date/time classes.

### (i) LocalDate (Date Only)

- Represents a **date** without time (format: `yyyy-MM-dd` ).
- **Thread-safe** and **immutable**.

#### Common Methods:

- `LocalDate.now()` → Gets the current date.
- `LocalDate.of(year, month, day)` → Creates a specific date.
- `LocalDate.parse("2025-04-04")` → Parses a date string.
- `isBefore()` , `isAfter()` , `isEqual()` , `compareTo()` → Comparison methods.

#### Example:

```
import java.time.LocalDate;

public class LocalDateExample {
    public static void main(String[] args) {
        LocalDate today = LocalDate.now();
        System.out.println("Current Date: " + today);

        LocalDate specificDate = LocalDate.of(2023, 2, 21);
        System.out.println("Specific Date: " + specificDate);

        LocalDate parsedDate = LocalDate.parse("2025-04-04");
        System.out.println("Parsed Date: " + parsedDate);
    }
}
```

### (ii) LocalTime (Time Only)

- Represents only **time** (format: `HH:mm:ss` ).
- **Immutable** and **thread-safe**.

### Common Methods:

- `LocalTime.now()` → Gets the current time.
- `LocalTime.of(hour, minute, second)` → Creates a specific time.

### Example:

```
import java.time.LocalTime;

public class LocalTimeExample {
    public static void main(String[] args) {
        LocalTime now = LocalTime.now();
        System.out.println("Current Time: " + now);

        LocalTime specificTime = LocalTime.of(14, 30, 15);
        System.out.println("Specific Time: " + specificTime);
    }
}
```

### (iii) LocalDateTime (Date + Time)

- Represents **both date and time** (format: `yyyy-MM-ddTHH:mm:ss`).
- **Thread-safe** and **immutable**.

### Common Methods:

- `LocalDateTime.now()` → Gets the current date-time.
- `LocalDateTime.of(date, time)` → Combines date and time.

### Example:

```
import java.time.LocalDateTime;

public class LocalDateTimeExample {
    public static void main(String[] args) {
        LocalDateTime now = LocalDateTime.now();
        System.out.println("Current Timestamp: " + now);

        LocalDateTime specificDateTime = LocalDateTime.of(2025, 4, 4, 12, 30, 45);
        System.out.println("Specific Date-Time: " + specificDateTime);
    }
}
```

### (iv) Formatting & Parsing Dates (DateTimeFormatter)

By default, `LocalDate` follows the `yyyy-MM-dd` format, but we can change it using `DateTimeFormatter`.

### Example:

```

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.Scanner;

public class DateFormatterExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter date (dd/MM/yyyy): ");
        String dateInput = sc.next();

        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        LocalDate formattedDate = LocalDate.parse(dateInput, formatter);

        System.out.println("Parsed Date: " + formattedDate);
        sc.close();
    }
}

```

## (v) Calculating Age (Using **Period** )

 **Example:**

```

import java.time.LocalDate;
import java.time.Period;

public class AgeCalculator {
    public static void main(String[] args) {
        LocalDate birthDate = LocalDate.of(1998, 5, 10);
        LocalDate today = LocalDate.now();

        Period age = Period.between(birthDate, today);
        System.out.println("Age: " + age.getYears() + " years, " +
            age.getMonths() + " months, " + age.getDays() + " days");
    }
}

```

## (vi) Adding & Subtracting Dates ( **plus()** , **minus()** )

 **Example:**

```

import java.time.LocalDate;

public class DateArithmetic {
    public static void main(String[] args) {
        LocalDate today = LocalDate.now();

        LocalDate nextWeek = today.plusWeeks(1);
    }
}

```

```

    LocalDate lastMonth = today.minusMonths(1);

    System.out.println("Today: " + today);
    System.out.println("Next Week: " + nextWeek);
    System.out.println("Last Month: " + lastMonth);
}
}

```

## Advantages of Java 8 Date/Time API

- ✓ **Immutable & Thread-Safe** – No need for external synchronization.
- ✓ **Easy Parsing & Formatting** – No need for `SimpleDateFormat`.
- ✓ **More Readable & Consistent API** – Uses clear class names like `LocalDate`, `LocalTime`.
- ✓ **Prevents Invalid Dates** – Throws `DateTimeParseException` for incorrect values.

## Summary

| Feature                      | Legacy ( <code>java.util.Date</code> ) | Java 8 ( <code>java.time</code> )          |
|------------------------------|----------------------------------------|--------------------------------------------|
| <b>Thread Safety</b>         | ✗ Not thread-safe                      | ✓ Thread-safe                              |
| <b>Immutability</b>          | ✗ Mutable                              | ✓ Immutable                                |
| <b>Formatting</b>            | Requires <code>SimpleDateFormat</code> | Uses <code>DateTimeFormatter</code>        |
| <b>Parsing</b>               | Complex & error-prone                  | Simple & reliable                          |
| <b>Invalid Date Handling</b> | Allows invalid values                  | Throws <code>DateTimeParseException</code> |

## String Class in Java

The `String` class in Java represents **immutable** character sequences, meaning any modification creates a new object instead of modifying the existing one.

### 1. Important String Constructors

| Constructor                                               | Description                                                                      |
|-----------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>String(byte[] bytes)</code>                         | Converts a byte array to a <code>String</code> .                                 |
| <code>String(char[] chars)</code>                         | Converts a character array to a <code>String</code> .                            |
| <code>String(byte[] bytes, int offset, int length)</code> | Converts a specific portion of a byte array to a <code>String</code> .           |
| <code>String(char[] chars, int offset, int length)</code> | Converts a specific portion of a character array to a <code>String</code> .      |
| <code>String(String s)</code>                             | Creates a new <code>String</code> object with the same value as <code>s</code> . |

#### 📌 Example:

```

byte[] byteArray = {65, 66, 67, 68}; // ASCII for "ABCD"
String strFromBytes = new String(byteArray);

```

```
System.out.println(strFromBytes); // Output: ABCD
```

## 2. Important String Methods

| Method                                                              | Description                                             |
|---------------------------------------------------------------------|---------------------------------------------------------|
| <code>charAt(int index)</code>                                      | Returns the character at the specified index.           |
| <code>compareTo(String s)</code>                                    | Compares two strings lexicographically.                 |
| <code>contains(CharSequence s)</code>                               | Checks if the string contains the specified sequence.   |
| <code>copyValueOf(char[] data)</code>                               | Converts a character array into a <code>String</code> . |
| <code>format(String format, Object... args)</code>                  | Returns a formatted string.                             |
| <code>valueOf(Object obj)</code>                                    | Converts an object to a string.                         |
| <code>getBytes()</code>                                             | Converts a <code>String</code> to a byte array.         |
| <code>toCharArray()</code>                                          | Converts a <code>String</code> to a character array.    |
| <code>toLowerCase()</code> / <code>toUpperCase()</code>             | Converts to lower/uppercase.                            |
| <code>indexOf(String s)</code>                                      | Returns the first occurrence of <code>s</code> .        |
| <code>lastIndexOf(String s)</code>                                  | Returns the last occurrence of <code>s</code> .         |
| <code>split(String regex)</code>                                    | Splits the string based on a regex pattern.             |
| <code>replace(CharSequence target, CharSequence replacement)</code> | Replaces occurrences of a substring.                    |
| <code>startsWith(String prefix)</code>                              | Checks if the string starts with the prefix.            |
| <code>endsWith(String suffix)</code>                                | Checks if the string ends with the suffix.              |
| <code>length()</code>                                               | Returns the length of the string.                       |
| <code>intern()</code>                                               | Returns a canonical representation of the string.       |

### Example:

```
String str = "Hello, Java!";
System.out.println(str.charAt(0));    // Output: H
System.out.println(str.toUpperCase()); // Output: HELLO, JAVA!
System.out.println(str.contains("Java")); // Output: true
```

## 3. equals() vs. == Comparison

### equals(Object o)

- Checks **content equality** (case-sensitive).
- Defined in `java.lang.Object` and **overridden in** `String`.

### Example:

```
String s1 = new String("hello");
String s2 = new String("hello");
```

```
System.out.println(s1.equals(s2)); // Output: true (Content Comparison)
System.out.println(s1 == s2);    // Output: false (Different Object References)
```

## 4. String Concatenation & Formatting

### concat()

- Combines two strings.



**Example:**

```
String s1 = "Hello";
String s2 = " World";
System.out.println(s1.concat(s2)); // Output: Hello World
```

## String Literals in Java

A **String literal** is a sequence of characters in **double quotes** ( `" "` ). It is stored in the **String Pool** for memory optimization.

### 1. Example & String Pool

```
String s1 = "Java";
String s2 = "Java"; // Same object in String Pool
String s3 = new String("Java"); // New object in Heap

System.out.println(s1 == s2); // true (same reference)
System.out.println(s1 == s3); // false (different reference)
```

#### Key Points:

- ✓ String literals are **automatically stored** in the **String Pool**.
- ✓ `new String("Java")` creates a **new object in Heap**.
- ✓ Use `.intern()` to move objects to the **String Pool**.

### 2. String Immutability

```
String s = "Hello";
s.concat(" World"); // No change (Immutable)
System.out.println(s); // Output: Hello

s = s.concat(" World"); // Now modified
System.out.println(s); // Output: Hello World
```

- ✓ Strings **cannot be modified**, new objects are created instead.

## Summary

- ✓ Use string literals ( `"Java"` ) instead of `new String("Java")` for efficiency.
- ✓ Strings are immutable (modifications create new objects).
- ✓ String Pool saves memory by reusing literals.

Would you like further simplification? 🚀

## String Formatting ( `printf` & `Formatter` class )

Java provides `printf` and `Formatter` for formatted output.

| Format Specifier                  | Description                                |
|-----------------------------------|--------------------------------------------|
| <code>%c</code>                   | Character                                  |
| <code>%b</code>                   | Boolean                                    |
| <code>%h</code>                   | Hexadecimal hashcode                       |
| <code>%s</code>                   | String                                     |
| <code>%d</code>                   | Integer                                    |
| <code>%f</code> , <code>%g</code> | Floating-point numbers                     |
| <code>%x</code>                   | Hexadecimal integer                        |
| <code>%n</code>                   | Newline                                    |
| <code>%tD</code>                  | Date ( <code>MM/DD/YY</code> )             |
| <code>%tT</code>                  | Time ( <code>HH:MM:SS</code> )             |
| <code>%tc</code>                  | Timestamp ( <code>Date &amp; Time</code> ) |

### 📌 Example:

```
int num = 25;
double price = 45.678;
System.out.printf("Number: %d, Price: %.2f%n", num, price);
// Output: Number: 25, Price: 45.68
```

## 5. Legacy Date/Time API ( `java.util.Date` )

### Constructors

| Constructor                          | Description                                             |
|--------------------------------------|---------------------------------------------------------|
| <code>Date()</code>                  | Creates a date object with the current system time.     |
| <code>Date(long milliseconds)</code> | Creates a date based on milliseconds from 1st Jan 1970. |

### 📌 Example:

```
import java.util.Date;

public class DateExample {
    public static void main(String[] args) {
        Date now = new Date();
        System.out.println(now); // Output: Current Date and Time
    }
}
```

```

        Date specificDate = new Date(10000000000L);
        System.out.println(specificDate);
    }
}

```

## Methods

| Method                         | Description                                   |
|--------------------------------|-----------------------------------------------|
| <code>before(Date d)</code>    | Checks if the date is before <code>d</code> . |
| <code>after(Date d)</code>     | Checks if the date is after <code>d</code> .  |
| <code>compareTo(Date d)</code> | Compares two dates.                           |

## 6. Date Parsing & Formatting ( `SimpleDateFormat` )

### Formatting Date → String

 **Example:**

```

import java.text.SimpleDateFormat;
import java.util.Date;

public class DateFormatExample {
    public static void main(String[] args) {
        Date date = new Date();
        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy HH:mm:ss");
        String formattedDate = sdf.format(date);
        System.out.println("Formatted Date: " + formattedDate);
    }
}

```

### Parsing String → Date

 **Example:**

```

import java.text.SimpleDateFormat;
import java.util.Date;

public class DateParseExample {
    public static void main(String[] args) throws Exception {
        String dateStr = "04-04-2025 15:30:00";
        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy HH:mm:ss");
        Date parsedDate = sdf.parse(dateStr);
        System.out.println("Parsed Date: " + parsedDate);
    }
}

```

## 7. GregorianCalendar Class

- Sub-class of `Calendar`
- Month range: `0-11` (Jan = 0, Dec = 11)
- Constructors:

```
GregorianCalendar(int year, int month, int day);
GregorianCalendar(int year, int month, int day, int hour, int minute, int second);
```

### Example:

```
import java.util.GregorianCalendar;
import java.util.Calendar;

public class CalendarExample {
    public static void main(String[] args) {
        GregorianCalendar gc = new GregorianCalendar(2025, Calendar.APRIL, 4);
        System.out.println("Year: " + gc.get(Calendar.YEAR));
        System.out.println("Month: " + (gc.get(Calendar.MONTH) + 1)); // Adjusting 0-based month
    }
}
```

## 8. Variable Arguments ( `var-args` `...` )

Allows methods to accept a variable number of arguments.

### Example:

```
public class VarArgsExample {
    public static void main(String[] args) {
        printNumbers(10, 20, 30);
        printNumbers(1, 2, 3, 4, 5);
    }

    static void printNumbers(int... nums) {
        for (int num : nums) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

## Summary

- ✓ String is immutable
- ✓ `equals()` checks content, `==` checks reference

- ✓ Legacy Date API is replaced by Java 8 `java.time` API
  - ✓ Use `SimpleDateFormat` for formatting/parsing dates
  - ✓ Var-args ( `...` ) allow flexible method parameters
- 

## Day 9

### Java `enum` (Enumeration) Overview

`enum` is a special Java keyword introduced in **JDK 1.5** that defines a **fixed set of constants**. It provides **type safety** and is more powerful than enums in C++.

---

#### 1. Why Use `enum` ?

- ✓ Defines a **fixed** set of related constants.
  - ✓ Provides **type safety** (avoids accidental mismatches).
  - ✓ Allows **iteration** over values.
  - ✓ Can have **constructors, methods, and fields**.
- 

#### 2. Basic `enum` Example

```
enum Direction {  
    EAST, WEST, NORTH, SOUTH  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Direction d = Direction.NORTH;  
        System.out.println(d); // Output: NORTH  
    }  
}
```

- ✓ Constants like `EAST`, `WEST`, etc., are implicitly `public static final`.
- 

#### 3. Enum with Fields & Methods

```
enum Direction {  
    EAST(0), WEST(180), NORTH(90), SOUTH(270);  
  
    private int angle;  
  
    Direction(int angle) { // Constructor (private by default)  
        this.angle = angle;  
    }  
  
    public int getAngle() { return angle; }  
}
```

```
public class Main {
    public static void main(String[] args) {
        System.out.println(Direction.EAST.getAngle()); // Output: 0
    }
}
```

- ✓ **Constructors in enums are always private.**
- ✓ **Each constant can have associated values and methods.**

## 4. Important `enum` Methods

```
Direction d = Direction.NORTH;
System.out.println(d.name()); // NORTH (Returns name of constant)
System.out.println(d.ordinal()); // 2 (Index position)
System.out.println(Direction.valueOf("EAST")); // EAST
```

- ✓ `.name()` → Returns the **name** of the enum constant.
- ✓ `.ordinal()` → Returns **index position** (starting from 0).
- ✓ `.valueOf("EAST")` → Converts **String to enum** (throws `IllegalArgumentException` for invalid values).
- ✓ `.values()` → Returns **all enum constants** as an array.

## 5. Enum Inside a Class

```
public class Test {
    enum Status { ACTIVE, INACTIVE }

    public static void main(String[] args) {
        System.out.println(Status.ACTIVE); // ACTIVE
    }
}
```

- ✓ Enums inside a class are **static by default**.

## 6. Enum vs Interface Constants

### ✗ Bad Approach (Using Interface for Constants)

```
interface MovieConstants {
    int AGE_MINOR = 16;
    int TKT_COST_GOLD = 200;
}
```

### ✓ Better Approach (Using `enum`)

```
enum TicketType {
    SILVER(100), GOLD(200), PLATINUM(300);
}
```

```
private int cost;
TicketType(int cost) { this.cost = cost; }
public int getCost() { return cost; }
}
```

✓ **Prevents accidental mismatches** (e.g., comparing `AGE_MINOR` with `TKT_COST_GOLD`).

## 7. Comparing Enums

```
Direction d1 = Direction.NORTH;
Direction d2 = Direction.SOUTH;

System.out.println(d1 == d2);    // false (Uses `==` for comparison)
System.out.println(d1.equals(d2)); // false
System.out.println(d1.compareTo(d2)); // -2 (Compares based on ordinal value)
```

✓ **Enums are singletons** → Always use `==` for comparison.

✓ `compareTo()` compares **ordinal values**.

## 8. Enum with Switch Case

```
Direction d = Direction.EAST;

switch (d) {
    case EAST:
        System.out.println("Move Right");
        break;
    case WEST:
        System.out.println("Move Left");
        break;
}
```

✓ **Switch works directly with enums**, unlike regular objects.

## Summary

- ✓ **Enums provide type safety for constants.**
- ✓ **They can have fields, constructors, and methods.**
- ✓ **Enums are implicitly `public static final`, singleton, and immutable.**
- ✓ **Best practice over using `interface` constants.**

## Nested Classes in Java

A **nested class** is a class that is declared **inside another class**. It helps in better code organization and encapsulation.

# 1. Types of Nested Classes in Java

## ◆ 1.1. Non-Static Nested Class (Inner Class)

- **Has access to all members** (including private) of the outer class.
- Requires an **instance of the outer class** to be created.

**Example:**

```
class Outer {
    private String message = "Hello from Outer class";

    class Inner { // Non-static nested class
        void display() {
            System.out.println(message); // Can access private members
        }
    }

    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner(); // Requires Outer instance
        inner.display(); // Output: Hello from Outer class
    }
}
```

✓ **Inner class can access private members of Outer class.**

✓ **Must be instantiated using an instance of Outer class.**

## ◆ 1.2. Static Nested Class

- Declared using the `static` keyword.
- **Does NOT require an instance of the outer class.**
- Can **only access static members** of the outer class.

**Example:**

```
class Outer {
    static String message = "Hello from Static Nested Class";

    static class StaticNested {
        void display() {
            System.out.println(message); // Only accesses static members
        }
    }

    public static void main(String[] args) {
        Outer.StaticNested nested = new Outer.StaticNested(); // No Outer instance needed
        nested.display(); // Output: Hello from Static Nested Class
    }
}
```

```
}  
}
```

- ✓ **Does NOT need an instance of Outer class.**
- ✓ **Can access only `static` members of Outer class.**

### ◆ 1.3. Method-Local Inner Class

- Defined inside a **method**.
- **Can only be used within that method.**
- **Cannot access non-final local variables** (before Java 8, they must be `final`).

**Example:**

```
class Outer {  
    void outerMethod() {  
        class LocalInner { // Method-local inner class  
            void display() {  
                System.out.println("Inside Method-Local Inner Class");  
            }  
        }  
  
        LocalInner localInner = new LocalInner();  
        localInner.display();  
    }  
  
    public static void main(String[] args) {  
        new Outer().outerMethod(); // Output: Inside Method-Local Inner Class  
    }  
}
```

- ✓ **Class exists only inside the method.**
- ✓ **Can only access `final` or effectively final local variables.**

### ◆ 1.4. Anonymous Inner Class

- **Has no name** and is used for **one-time use**.
- Used when you need a **class with minimal modifications**.
- Created **at the point of instantiation**.

**Example:** Implementing an interface

```
abstract class Greeting {  
    abstract void sayHello();  
}  
  
public class Test {  
    public static void main(String[] args) {
```

```

    Greeting g = new Greeting() { // Anonymous Inner Class
        void sayHello() {
            System.out.println("Hello from Anonymous Inner Class");
        }
    };

    g.sayHello(); // Output: Hello from Anonymous Inner Class
}

```

✓ Used when you **don't want to create a separate class**.

✓ **Directly creates an instance while defining the class.**

## 2. Key Differences

| Feature                            | Non-Static Inner Class | Static Nested Class | Method-Local Inner Class | Anonymous Inner Class |
|------------------------------------|------------------------|---------------------|--------------------------|-----------------------|
| <b>Requires Outer Instance?</b>    | ✓ Yes                  | ✗ No                | ✓ Yes                    | ✓ Yes                 |
| <b>Can Access Private Members?</b> | ✓ Yes                  | ✗ No                | ✓ Yes                    | ✓ Yes                 |
| <b>Can Have Static Members?</b>    | ✗ No                   | ✓ Yes               | ✗ No                     | ✗ No                  |
| <b>Used For?</b>                   | Grouping logic         | Helper classes      | Method-specific logic    | One-time use          |

## Summary

✓ **Nested classes** improve encapsulation and organization.

✓ **Inner classes** can access **outer class members directly**.

✓ **Static nested classes** work like independent classes.

✓ **Method-local inner classes** are confined to a method.

✓ **Anonymous inner classes** are used for quick one-time implementation.

Would you like more examples or a specific topic explained further? 🚀

## Wrapper Classes in Java

Wrapper classes provide an **object representation** for primitive data types.

### 1 Why Do We Need Wrapper Classes?

✓ **Allows primitive types to be stored in collections** (e.g., `ArrayList`, `HashMap`).

✓ **Provides utility methods** (e.g., `parseInt()`, `isDigit()`, `isWhitespace()`).

Example:

```
ArrayList<Integer> list = new ArrayList<>();  
list.add(10); // Allowed due to auto-boxing
```

## 2 What Are Wrapper Classes?

A **wrapper class** is an object representation of a primitive type.

| Primitive Type | Wrapper Class |
|----------------|---------------|
| byte           | Byte          |
| short          | Short         |
| int            | Integer       |
| long           | Long          |
| float          | Float         |
| double         | Double        |
| char           | Character     |
| boolean        | Boolean       |

## 3 Inheritance Hierarchy

```
java.lang.Object  
|  
├── Character (for char)  
├── Boolean (for boolean)  
└── Number  
    ├── Byte  
    ├── Short  
    ├── Integer  
    ├── Long  
    ├── Float  
    └── Double
```

- ✓ All wrapper classes extend **Object**.
- ✓ All numeric wrappers extend **Number** (except **Character** and **Boolean**).

## 4 Boxing & Unboxing (Before JDK 1.5)

**Boxing:** (Primitive → Wrapper)

```
Integer i1 = new Integer(100); // Explicit Boxing  
Double d1 = new Double(10.5);
```

**Unboxing:** (Wrapper → Primitive)

```
int num = i1.intValue(); // Explicit Unboxing
double val = d1.doubleValue();
```

## 5 Auto-Boxing & Auto-Unboxing (JDK 1.5+)

**Auto-Boxing:** (Primitive → Wrapper Automatically)

```
Integer i1 = 100; // Compiler automatically converts int → Integer
Double d1 = 12.34; // Compiler converts double → Double
```

**Auto-Unboxing:** (Wrapper → Primitive Automatically)

```
int num = i1; // Compiler converts Integer → int
double val = d1; // Compiler converts Double → double
```

✓ **Works seamlessly with operations:**

```
Integer i = 10;
i++; // Auto-unboxing (Integer → int), Increment, Auto-boxing (int → Integer)
System.out.println(i); // Output: 11
```

## 6 Common Methods in Wrapper Classes

| Method                             | Description                                         | Example                                          |
|------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| <code>parseInt(String s)</code>    | Converts <code>String</code> to <code>int</code>    | <code>Integer.parseInt("123") → 123</code>       |
| <code>parseDouble(String s)</code> | Converts <code>String</code> to <code>double</code> | <code>Double.parseDouble("3.14") → 3.14</code>   |
| <code>valueOf(String s)</code>     | Converts <code>String</code> to wrapper object      | <code>Integer.valueOf("10") → Integer(10)</code> |
| <code>intValue()</code>            | Converts <code>Integer</code> to <code>int</code>   | <code>i1.intValue()</code>                       |
| <code>doubleValue()</code>         | Converts <code>Double</code> to <code>double</code> | <code>d1.doubleValue()</code>                    |
| <code>isDigit(char c)</code>       | Checks if char is a digit ( 0-9 )                   | <code>Character.isDigit('5') → true</code>       |
| <code>isWhitespace(char c)</code>  | Checks if char is whitespace                        | <code>Character.isWhitespace(' ') → true</code>  |

## 7 Examples

### Example 1: Using Auto-Boxing & Auto-Unboxing

```
public class WrapperDemo {
    public static void main(String[] args) {
        Integer i = 100; // Auto-boxing
        int num = i; // Auto-unboxing

        System.out.println("Integer Object: " + i);
        System.out.println("Primitive int: " + num);
    }
}
```

```
}  
}
```

#### ✓ Output:

```
Integer Object: 100  
Primitive int: 100
```

## Example 2: Using Wrapper Methods

```
public class WrapperMethods {  
    public static void main(String[] args) {  
        String str = "123";  
        int num = Integer.parseInt(str); // Convert String to int  
        Double d = Double.valueOf("12.34"); // Convert String to Double object  
  
        System.out.println(num + 10); // 133  
        System.out.println(d * 2); // 24.68  
    }  
}
```

## ◆ Key Takeaways

- ✓ Wrapper classes **store primitive values as objects**.
- ✓ **Auto-boxing** (primitive → wrapper) and **Auto-unboxing** (wrapper → primitive) were introduced in **JDK 1.5**.
- ✓ Used in **collections, utility methods, and type safety**.
- ✓ Provides **useful methods** like `parseInt()`, `valueOf()`, and `isDigit()`.

## Day 10

## Java Collections Framework (JCF)

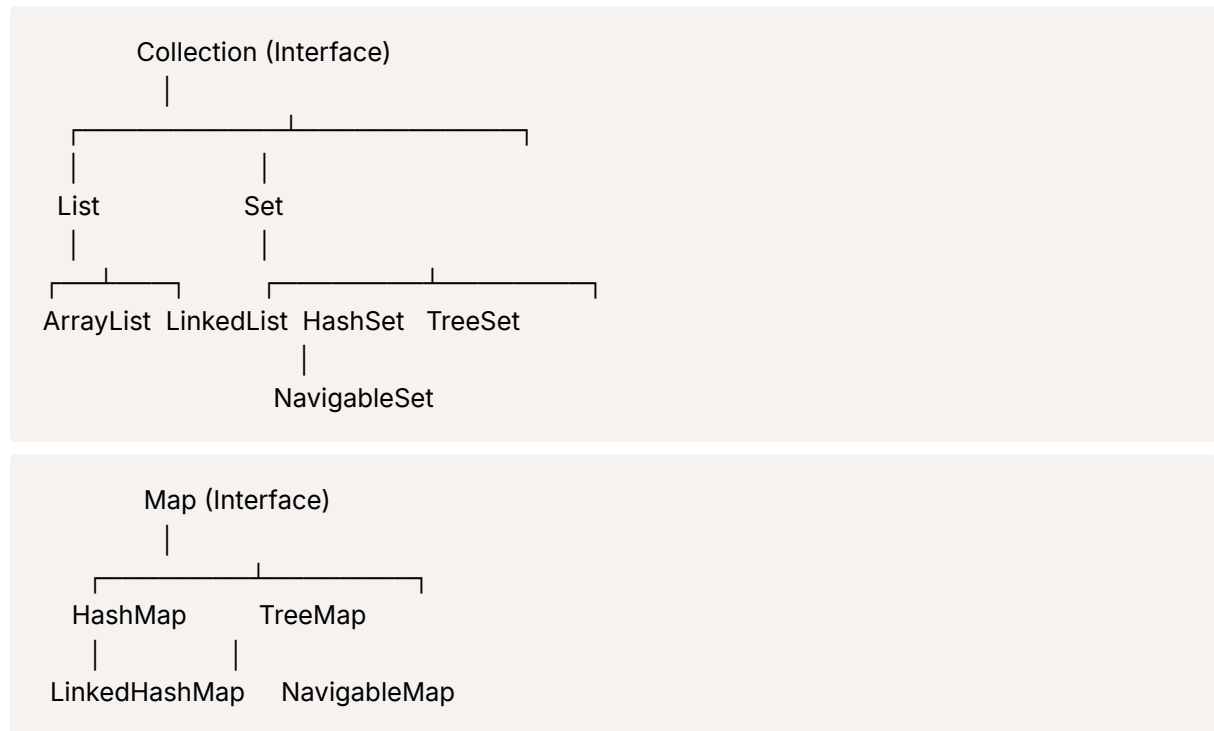
The **Java Collections Framework (JCF)** is a set of classes and interfaces in `java.util` package that provides **dynamic data structures** like **lists, sets, queues, and maps**.

### 1 Why Java Collections?

- ✓ **Reduces coding effort** – Provides ready-made, optimized data structures.
- ✓ **Improves performance** – Uses efficient algorithms internally.
- ✓ **Type safety** – Generic support ensures type correctness.
- ✓ **Dynamic in size** – Unlike arrays, collections grow as needed.

✓ **Useful utilities** – Sorting, searching, and thread safety features.

## 2 Java Collection Framework Hierarchy



## 3 Core Interfaces in Java Collections

| Interface | Description                                 | Implementations                       |
|-----------|---------------------------------------------|---------------------------------------|
| List      | Ordered collection (allows duplicates).     | ArrayList, LinkedList, Vector, Stack  |
| Set       | Unordered collection (no duplicates).       | HashSet, TreeSet, LinkedHashSet       |
| Queue     | FIFO-based collection.                      | LinkedList, PriorityQueue, ArrayDeque |
| Deque     | Double-ended queue.                         | ArrayDeque, LinkedList                |
| Map       | Stores key-value pairs (no duplicate keys). | HashMap, TreeMap, LinkedHashMap       |

## 4 List Interface (Ordered, Allows Duplicates)

### ✓ ArrayList (Dynamic Array)

- ✓ Fast random access (  $O(1)$  lookup).
- ✓ Slow insertions/deletions (  $O(n)$  ).

```
List<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
System.out.println(list); // [Apple, Banana]
```

### ✓ LinkedList (Doubly Linked List)

- ✓ Fast insertions/deletions (  $O(1)$  ).
- ✓ Slow random access (  $O(n)$  ).

```
List<Integer> list = new LinkedList<>();  
list.add(10);  
list.add(20);  
System.out.println(list); // [10, 20]
```

### ✓ **Vector** (Thread-Safe)

- ✓ Synchronized version of `ArrayList`.

```
List<Integer> vector = new Vector<>();  
vector.add(1);  
vector.add(2);
```

## 5 Set Interface (Unique Elements, No Duplicates)

### ✓ **HashSet** (Unordered, Uses Hashing)

- ✓ Fast lookups (  $O(1)$  ).
- ✓ No ordering.

```
Set<String> set = new HashSet<>();  
set.add("Java");  
set.add("Python");  
System.out.println(set); // [Python, Java] (unordered)
```

### ✓ **TreeSet** (Sorted, Balanced Tree)

- ✓ Elements are stored in **sorted order**.
- ✓ Slower operations (  $O(\log n)$  ).

```
Set<Integer> set = new TreeSet<>();  
set.add(5);  
set.add(2);  
set.add(10);  
System.out.println(set); // [2, 5, 10] (sorted)
```

## 6 Map Interface (Key-Value Pairs)

### ✓ **HashMap** (Unordered, Fast)

- ✓ Key-value pairs (  $O(1)$  lookup).
- ✓ No order maintained.

```
Map<String, Integer> map = new HashMap<>();
map.put("Apple", 100);
map.put("Banana", 50);
System.out.println(map); // {Apple=100, Banana=50}
```

### ✓ **TreeMap** (Sorted by Key)

✓ Maintains **ascending order** of keys.

```
Map<Integer, String> map = new TreeMap<>();
map.put(3, "C");
map.put(1, "A");
map.put(2, "B");
System.out.println(map); // {1=A, 2=B, 3=C}
```

## 7 Queue & Deque (FIFO, LIFO)

### ✓ **PriorityQueue** (Sorted Queue)

✓ Elements are ordered based on **priority**.

```
Queue<Integer> pq = new PriorityQueue<>();
pq.add(30);
pq.add(10);
pq.add(20);
System.out.println(pq.poll()); // 10 (smallest removed first)
```

### ✓ **ArrayDeque** (Double-ended Queue)

✓ Faster than **Stack** and **LinkedList** for **push/pop operations**.

```
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("Start");
deque.addLast("End");
System.out.println(deque); // [Start, End]
```

## 8 Sorting Collections

```
List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 2, 8, 1));
Collections.sort(numbers);
System.out.println(numbers); // [1, 2, 5, 8]
```

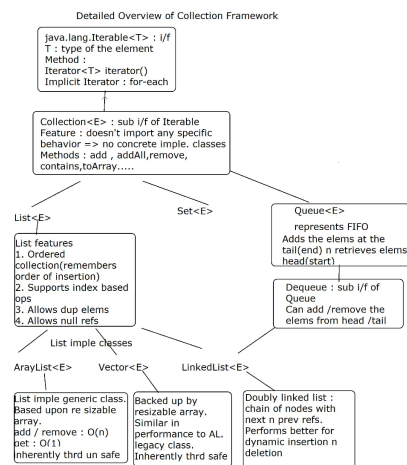
## 9 Thread-Safe Collections

```
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
```

## ◆ Key Takeaways

- ✓ **List** → Ordered, allows duplicates ( `ArrayList` , `LinkedList` ).
- ✓ **Set** → Unique elements, no duplicates ( `HashSet` , `TreeSet` ).
- ✓ **Map** → Key-value pairs ( `HashMap` , `TreeMap` ).
- ✓ **Queue** → FIFO, **Deque** → Double-ended queue ( `PriorityQueue` , `ArrayDeque` ).

Would you like more details or examples? 🚀



This is a great summary of the `List<E>` interface in Java! Here's a **cleaned-up, structured, and slightly optimized** version for quick reference:

## List in Java

### 1 Features of List

- ✓ **Ordered Collection** – Maintains insertion order.
- ✓ **Allows null values.**
- ✓ **Allows duplicates.**
- ✓ **Supports index-based operations.**

### 2 Important Methods in List

#### 📌 1. Adding Elements

```
boolean add(E e) // Appends element at the end
void add(int index, E e) // Inserts at specified index
```

```
void addAll(Collection<E> c) // Adds all elements from another collection
```

✓ `addAll` supports:

- `ArrayList` , `LinkedList` , `Vector` ✓
- `HashSet` , `LinkedHashSet` , `TreeSet` ✓
- `HashMap` , `LinkedHashMap` , `TreeMap` ✗ (Not a Collection)

## 📌 2. Retrieving Elements

```
E get(int index) // Returns element at index
```

🚫 Throws `IndexOutOfBoundsException` if index is invalid.

## 📌 3. Displaying Elements

✓ `toString()` method

✓ **Iteration Methods:**

- **Iterator** ( `list.iterator()` )
- **For-each loop**
- **For loop** ( `list.get(i)` )

## 📌 4. Searching for Elements

```
boolean contains(Object o) // Returns true if element is found  
int indexOf(Object o) // Returns first occurrence index  
int lastIndexOf(Object o) // Returns last occurrence index
```

💡 **Uses `equals()` method for comparison.**

✓ Best practice: Override `equals()` method for custom objects.

## 📌 5. Modifying Elements

```
E set(int index, E newElem) // Replaces element at index and returns old value
```

## 📌 6. Removing Elements

```
E remove(int index) // Removes element by index  
boolean remove(Object o) // Removes first occurrence of specified element
```

💡 Returns `true` if removal is successful, `false` otherwise.

## 📌 7. Conversion from Collection to Array

```
Object[] toArray() // Returns array of Objects
T[] toArray(T[] array) // Returns array of specified type
```

### 3 List Implementation Classes

#### ✓ **ArrayList<E>** (Growable Array)

- ✓ **Fast random access** (  $O(1)$  ).
- ✓ **Resizes dynamically** (Default capacity = 10).
- ✓ **Slower insertions/removals** (  $O(n)$  ).

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Python");
System.out.println(list); // [Java, Python]
```

#### ArrayList Constructors

```
ArrayList<E>() // Default constructor (Capacity = 10)
ArrayList<E>(int initCapacity) // Custom capacity
ArrayList<E>(Collection<E> c) // Creates list from another collection
```

### ◆ Key Takeaways

- ✓ **List<E>** is **ordered**, **allows duplicates**, and **supports indexing**.
- ✓ **ArrayList<E>** is the most commonly used implementation.
- ✓ **Always override** `equals()` **for searching custom objects**.

## Java Generics: A Quick Guide

### 1 What are Generics?

- ✓ Introduced in **Java SE 5**.
- ✓ Represents **parameterized types** ( `<T>` ).
- ✓ Used in **classes, interfaces, methods, and constructors**.
- ✓ **Before Generics**: Used `Object` references (Type casting required).

### 2 Syntax & Example

💡 **Similar to C++ templates (angle brackets `<>`), but different for nested collections.**

```
class MyGeneric<T> { // Generic class with one type parameter
    private T ref;
```

```
}
```

```
class MyGeneric<T, U> { // Generic class with multiple type parameters
    private T ref1;
    private U ref2;
}
```

#### ✓ Example Usage

```
MyGeneric<Integer, String> obj = new MyGeneric<>();
```

### 3 Advantages of Generics

- ✓ **Type Safety @ Compile Time** – No `ClassCastException`.
- ✓ **Automatic Type Casting** – No need for explicit type conversion.
- ✓ **Better Code Reusability** – Works with **any** reference type ( `String` , `Integer` , `List<T>` , etc.).

**Example:**

```
// Without Generics (Type Casting Required)
List list = new ArrayList();
list.add("Java");
String s = (String) list.get(0); // Explicit casting needed

// With Generics (No Casting)
List<String> list2 = new ArrayList<>();
list2.add("Java");
String s2 = list2.get(0); // No casting required
```

### 4 Type Erasure in Java

- 💡 **Generic type information exists only at compile time.**
- 💡 **JVM removes type parameters during compilation** (i.e., `ArrayList<T>` becomes `ArrayList` ).

```
public class Holder<T> {
    private T ref;
}
```

#### ⬇ After Type Erasure ⬇

```
public class Holder {
    private Object ref;
}
```

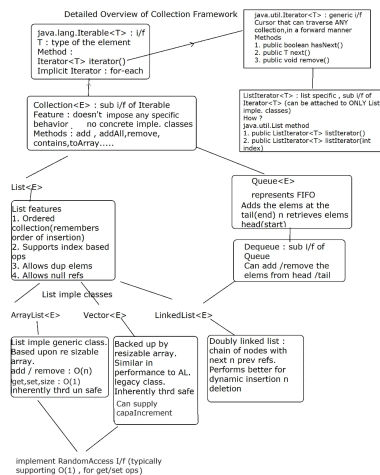
🚫 **No runtime type information for generics!**

## 5 Key Takeaways

- ✓ Generics ensure **compile-time safety** and eliminate **type-casting**.
- ✓ **No performance overhead** (handled at compile time).
- ✓ **JVM uses Type Erasure**, removing type parameters from bytecode.

Would you like some **examples on generic methods or bounded types**? 🚀

## Day 11



## Sorting in Java – Quick Guide

Java provides **two ways** to sort objects:

### 1 Natural Ordering (Comparable Interface)

- 📌 Based on `java.lang.Comparable<T>` (generic interface).
- 📌 Used when objects have a **default sorting order** (e.g., sorting `Customer` by `ID`).

### Steps to Implement Comparable

- ✓ **Step 1:** Implement `Comparable<T>` in your class.
- ✓ **Step 2:** Override `compareTo(T anotherObject)`.
- ✓ **Step 3:** Use `Collections.sort(List<T> list)`.

```
class Customer implements Comparable<Customer> {  
    private int id;  
    private String name;  
  
    public Customer(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```

    }

    // Natural ordering: Sort by ID
    @Override
    public int compareTo(Customer other) {
        return Integer.compare(this.id, other.id);
    }
}

```

### Sorting a list of customers:

```

List<Customer> customers = new ArrayList<>();
customers.add(new Customer(3, "John"));
customers.add(new Customer(1, "Alice"));
customers.add(new Customer(2, "Bob"));

Collections.sort(customers); // Uses compareTo() method

```

✅ **Internally uses TimSort** (optimized merge + insertion sort).

⚠️ **Limitation: Only one sorting criteria allowed** (e.g., can't switch to name-based sorting).

## 2 Custom Ordering (Comparator Interface)

📌 Based on `java.util.Comparator<T>`.

📌 Allows sorting on **multiple criteria** (e.g., sort `Customer` by `name`, then `ID`).

### Steps to Implement Comparator

✅ **Step 1:** Create a separate class implementing `Comparator<T>`.

✅ **Step 2:** Override `compare(T o1, T o2)`.

✅ **Step 3:** Use `Collections.sort(List<T> list, Comparator<T> comp)`.

```

class NameComparator implements Comparator<Customer> {
    @Override
    public int compare(Customer c1, Customer c2) {
        return c1.name.compareTo(c2.name); // Sort by name (Alphabetically)
    }
}

```

### Sorting using a custom comparator:

```

Collections.sort(customers, new NameComparator()); // Sorts by name

```

### 💡 Alternative: Anonymous Inner Class (Shorter Code)

```

Collections.sort(customers, new Comparator<Customer>() {
    @Override

```

```
public int compare(Customer c1, Customer c2) {
    return c1.name.compareTo(c2.name);
}
});
```

## 🔥 Lambda Expression (Java 8+)

```
customers.sort((c1, c2) → c1.name.compareTo(c2.name)); // One-liner!
```

## 🚀 Key Takeaways

- ✓ Use **Comparable** ( `compareTo()` ) for default sorting.
- ✓ Use **Comparator** ( `compare()` ) for custom/multiple sorting orders.
- ✓ Java 8+ allows cleaner sorting using Lambdas!

## Iterator in Java

📌 **Iterator** is an interface in `java.util` package used to traverse (iterate over) collections like `ArrayList`, `HashSet`, etc.

📌 It provides **sequential access** to elements **without exposing** underlying data structure.

📌 **Introduced in Java 2** (JDK 1.2).

### 1 Why Use Iterator?

- ✓ Works on **all collection types** ( `List`, `Set`, `Queue`, etc.).
- ✓ Avoids **ConcurrentModificationException** (compared to using `for-each` loop).
- ✓ **Removes elements safely** during iteration (unlike `for-each` ).
- ✓ Supports **forward iteration only** ( `Iterator` does not support backward iteration).

### 2 Iterator Methods

Iterator interface has **three main methods**:

| Method                         | Description                                           |
|--------------------------------|-------------------------------------------------------|
| <code>boolean hasNext()</code> | Returns <code>true</code> if there are more elements. |
| <code>E next()</code>          | Returns the next element.                             |
| <code>void remove()</code>     | Removes the last returned element (optional).         |

### 3 Example – Using Iterator with ArrayList

```
import java.util.*;

public class IteratorExample {
    public static void main(String[] args) {
```

```

List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Charlie");

// Get Iterator
Iterator<String> itr = names.iterator();

// Iterate using Iterator
while (itr.hasNext()) {
    System.out.println(itr.next());
}
}
}

```

#### ✓ Output:

```

Alice
Bob
Charlie

```

## 4 Removing Elements Using Iterator

```

Iterator<String> itr = names.iterator();
while (itr.hasNext()) {
    if (itr.next().equals("Bob")) {
        itr.remove(); // Removes "Bob"
    }
}
}

```

⚠ DO NOT use `names.remove("Bob")` inside `for-each` , it will throw `ConcurrentModificationException` !

## 5 ListIterator – For Lists Only

📌 `ListIterator` is a subinterface of `Iterator` that allows **both forward & backward traversal**.

### Methods of ListIterator

| Method                     | Description                                                                            |
|----------------------------|----------------------------------------------------------------------------------------|
| <code>hasPrevious()</code> | Returns <code>true</code> if there is a previous element.                              |
| <code>previous()</code>    | Returns the previous element.                                                          |
| <code>add(E e)</code>      | Adds an element at current position.                                                   |
| <code>set(E e)</code>      | Replaces the last element returned by <code>next()</code> or <code>previous()</code> . |

### Example: Using ListIterator

```
List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob", "Charlie"));
ListIterator<String> listItr = names.listIterator(names.size());

while (listItr.hasPrevious()) {
    System.out.println(listItr.previous()); // Prints in reverse order
}
```

#### ✓ Output:

```
Charlie
Bob
Alice
```

## 6 Enumeration (Legacy) vs Iterator vs ListIterator

| Feature              | Enumeration                                                         | Iterator                                                      | ListIterator                                                    |
|----------------------|---------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------|
| Can traverse?        | ✓ Forward only                                                      | ✓ Forward only                                                | ✓ Forward & Backward                                            |
| Can remove elements? | ✗ No                                                                | ✓ Yes                                                         | ✓ Yes                                                           |
| Can modify elements? | ✗ No                                                                | ✗ No                                                          | ✓ Yes (using <code>set()</code> )                               |
| Used with            | Legacy collections ( <code>Vector</code> , <code>Hashtable</code> ) | Any Collection ( <code>List</code> , <code>Set</code> , etc.) | Only Lists ( <code>ArrayList</code> , <code>LinkedList</code> ) |

## 7 Java 8+ Alternative – `forEach()` + Lambda

```
names.forEach(name → System.out.println(name));
```

#### ✓ Simpler & shorter than Iterator!

### Key Takeaways

- ✓ Use **Iterator** for safe removal & traversal of elements.
- ✓ Use **ListIterator** for **bi-directional traversal** & modifications.
- ✓ Use **Java 8+** `forEach()` with Lambdas for simple iteration.

Would you like examples of **fail-fast vs fail-safe iterators**? 😊

## Day 12

### ArrayList vs LinkedList in Java

Both `ArrayList` and `LinkedList` implement the **List** interface, but they differ in **performance, memory usage, and use cases**.

## 🔑 Key Differences Between ArrayList and LinkedList

| Feature                                | ArrayList                                               | LinkedList                                               |
|----------------------------------------|---------------------------------------------------------|----------------------------------------------------------|
| Search ( <code>get(int index)</code> ) | ✅ <b>Fast</b> → <code>O(1)</code> (Direct index access) | ❌ <b>Slow</b> → <code>O(n)</code> (Sequential traversal) |
| Insertion (add at index, middle)       | ❌ <b>Slow</b> → <code>O(n)</code> (Elements shift)      | ✅ <b>Fast</b> → <code>O(1)</code> (Pointer change)       |
| Deletion (remove at index, middle)     | ❌ <b>Slow</b> → <code>O(n)</code> (Elements shift)      | ✅ <b>Fast</b> → <code>O(1)</code> (Pointer change)       |
| Memory Usage                           | ✅ <b>Less</b> → Stores only data                        | ❌ <b>More</b> → Stores data + two pointers (prev, next)  |
| Iteration Performance                  | ✅ <b>Fast</b>                                           | ❌ <b>Slow</b>                                            |
| Random Access                          | ✅ <b>Supports</b> <code>RandomAccess</code> Interface   | ❌ <b>Does not support</b> <code>RandomAccess</code>      |
| Best Use Case                          | Search-heavy applications                               | Frequent insertions/deletions in middle                  |

## 🔍 When to Use What?

### ✅ Use `ArrayList` when:

- You need **fast random access** ( `get(int index)` ).
- You have **more search operations** than insertions/deletions.
- You have limited memory and want efficient storage.

### ✅ Use `LinkedList` when:

- You need **fast insertions/deletions in the middle**.
- Your application follows **FIFO (Queue) or LIFO (Stack)**.
- You don't need frequent **random access**.

## 💡 Quick Example

```
// ArrayList Example (Fast Search)
ArrayList<Integer> arrayList = new ArrayList<>();
arrayList.add(10);
arrayList.add(20);
System.out.println(arrayList.get(1)); // O(1) Fast access

// LinkedList Example (Fast Insertion)
LinkedList<Integer> linkedList = new LinkedList<>();
linkedList.addFirst(10); // O(1)
linkedList.addLast(20); // O(1)
System.out.println(linkedList.removeFirst()); // O(1)
```

Would you like a diagram to visualize the differences? 😊

## Using `LinkedList` as a Queue (FIFO)

When using `LinkedList` as a **queue**, elements follow the **First-In-First-Out (FIFO)** principle.

- **Elements are added at the end** of the queue.  `offerLast(E e)` or `addLast(E e)`
- **Elements are removed from the front** of the queue.  `pollFirst()` or `removeFirst()`

### Example of LinkedList as a Queue

```
LinkedList<String> queue = new LinkedList<>();

// Enqueue (Add elements to the queue)
queue.offerLast("A");
queue.offerLast("B");
queue.offerLast("C");

// Dequeue (Remove elements from the queue)
System.out.println(queue.pollFirst()); // Output: A
System.out.println(queue.pollFirst()); // Output: B
System.out.println(queue.pollFirst()); // Output: C
```

#### Methods to use for Queue:

- `offerLast(E e)` → Add element at the end
- `pollFirst()` → Remove element from the front
- `peekFirst()` → Retrieve first element without removing

### Using `LinkedList` as a Stack (LIFO)

When using `LinkedList` as a **stack**, elements follow the **Last-In-First-Out (LIFO)** principle.

- **Elements are pushed at the beginning**.  `push(E e)` or `addFirst(E e)`
- **Elements are popped from the beginning**.  `pop()` or `removeFirst()`

### Example of LinkedList as a Stack

```
LinkedList<String> stack = new LinkedList<>();

// Push (Add elements to the stack)
stack.push("A");
stack.push("B");
stack.push("C");

// Pop (Remove elements from the stack)
System.out.println(stack.pop()); // Output: C
System.out.println(stack.pop()); // Output: B
System.out.println(stack.pop()); // Output: A
```

#### Methods to use for Stack:

- `push(E e)` → Add element at the top

- `pop()` → Remove and return the top element
- `peekFirst()` → Retrieve top element without removing

## Quick Summary

| Use Case            | Principle          | Add Method                                           | Remove Method                                    |
|---------------------|--------------------|------------------------------------------------------|--------------------------------------------------|
| <b>Queue</b> (FIFO) | First-In-First-Out | <code>offerLast(E e)</code>                          | <code>pollFirst()</code>                         |
| <b>Stack</b> (LIFO) | Last-In-First-Out  | <code>push(E e)</code> or <code>addFirst(E e)</code> | <code>pop()</code> or <code>removeFirst()</code> |

Would you like me to add a diagram for better understanding? 😊

## What is Hashing?

**Hashing** is a process of converting **arbitrary-sized data** into a **fixed-sized value (hash code)** using a **mathematical function (hash function)**.

It is widely used in **cryptography, data storage, and retrieval (like HashMap, HashSet in Java)**.

## Key Points About Hashing

- ✓ **Transforms input data** into a **fixed-size hash code**
- ✓ Used in **data structures (HashMap, HashSet)** and **security (password encryption, digital signatures, UUIDs)**
- ✓ **Fast lookups** → Used in **hash tables** for efficient searching
- ✓ **One-way function** → Hard to reverse a hash to get original data
- ✓ **Collision Handling** → A good **hash function reduces collisions** (when two inputs produce the same hash)

## Use Cases of Hashing

-  **Security:** Encrypting passwords using **SHA-256, SHA-512**
-  **Data Storage:** Hash-based indexing in **databases**
-  **Fast Searching:** HashMap in Java for **O(1) lookup time**
-  **Unique Identifiers:** Generating **UUIDs**

## Example: Hashing in Java

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class HashingExample {
    public static void main(String[] args) throws NoSuchAlgorithmException {
        String password = "MySecurePassword";

        // Create SHA-256 Hash
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(password.getBytes());
```

```

// Convert byte array to hexadecimal
StringBuilder hexHash = new StringBuilder();
for (byte b : hash) {
    hexHash.append(String.format("%02x", b));
}

System.out.println("Hashed Password: " + hexHash.toString());
}
}

```

#### Output Example:

```
Hashed Password: 5e884898da28047151d0e56f8dc6292773603d0d6aabbddc...
```

## Hashing-Based Data Structures in Java

Hashing-based data structures like **HashSet**, **HashMap**, and **HashTable** use **hash functions** to provide **fast O(1) lookups, insertions, and deletions**.

## How HashSet Works in Java

### Steps for Creating a HashSet

#### 1 Override `hashCode()` and `equals()` in the stored object type.

- `hashCode()` : Returns an `int`, determining the object's bucket location.
- `equals(Object obj)` : Ensures no duplicate elements exist.

#### 2 Follow the Hashing Contract:

- If `obj1.equals(obj2) == true`, then `obj1.hashCode() == obj2.hashCode()`. 
- If `obj1.equals(obj2) == false`, `obj1.hashCode()` may or may not be equal to `obj2.hashCode()`.

*(Different objects can have the same hash code, but identical objects must have the same hash code.)*

## How Does HashSet Ensure Constant Time Performance?

### Concept of Load Factor & Rehashing

- Every **HashSet** has a **capacity** and a **load factor** (default **0.75**).
- When **size > capacity × load factor**, **rehashing** occurs:
  - ✓ A **new larger hash table** is created (typically **double the size**).
  - ✓ **All elements are reinserted** based on their **new hash codes**.
  - ✓ This **reduces hash collisions** and maintains **O(1) performance**.

## Why No Duplicate Elements in HashSet?

◆ Ensured by the `hashCode()` and `equals()` contract

- ✓ If two elements are equal ( `equals() == true` ), they must have the same hash code.
- ✓ If hash codes match but elements are different, `equals()` checks the objects further.
- ✓ If an existing object is found equal to the new one, the new object is not added.

## Example: HashSet with Custom Class

```
import java.util.HashSet;
import java.util.Objects;

class Employee {
    int id;
    String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Employee emp = (Employee) obj;
        return id == emp.id && name.equals(emp.name);
    }
}

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<Employee> employees = new HashSet<>();
        employees.add(new Employee(101, "Alice"));
        employees.add(new Employee(102, "Bob"));
        employees.add(new Employee(101, "Alice")); // Duplicate, will not be added.

        System.out.println("Total Employees: " + employees.size()); // Output: 2
    }
}
```

## Explanation

- 1 Override `hashCode()` & `equals()` to ensure no duplicates.

2 The second "Alice" (ID: 101) is considered **duplicate**, so **not added**.

## Summary: HashSet vs HashMap vs HashTable

| Feature                    | HashSet                                | HashMap                                                         | HashTable                                  |
|----------------------------|----------------------------------------|-----------------------------------------------------------------|--------------------------------------------|
| Stores                     | Only keys (no values)                  | Key-Value pairs                                                 | Key-Value pairs                            |
| Allows <code>null</code> ? | Yes (only one <code>null</code> )      | Yes ( <code>null</code> key, multiple <code>null</code> values) | ✗ No <code>null</code> keys or values      |
| Thread-Safe?               | ✗ No                                   | ✗ No (use <code>ConcurrentHashMap</code> for thread safety)     | ✓ Yes (synchronized)                       |
| Performance                | Fast (O(1) for insert, delete, lookup) | Fast (O(1) lookup)                                              | Slower than HashMap due to synchronization |
| Ordering                   | ✗ Unordered                            | ✗ Unordered                                                     | ✗ Unordered                                |

## Day 13

### Java `Map` Interface Overview

The `Map` interface in Java **stores key-value pairs** and provides **fast lookup, insertion, and deletion** using **hashing** (for `HashMap` ) or **tree-based ordering** (for `TreeMap` ).

### HashMap Implementation

- ◆ `HashMap<K, V>` is a widely used implementation of `Map` .
- ◆ It allows **one `null` key** and **multiple `null` values**.
- ◆ Provides **O(1) lookup time** for keys using hashing.

### HashMap Constructors

| Constructor                                                 | Description                                                                                     |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>HashMap()</code>                                      | Creates an empty HashMap with an <b>initial capacity of 16</b> and <b>load factor of 0.75</b> . |
| <code>HashMap(int initCapacity)</code>                      | Creates a HashMap with a <b>specified initial capacity</b> .                                    |
| <code>HashMap(int initCapacity, float loadFactor)</code>    | Creates a HashMap with a specified capacity and <b>custom load factor</b> .                     |
| <code>HashMap(Map&lt;? extends K, ? extends V&gt; m)</code> | Creates a HashMap <b>populated with elements from another map</b> .                             |

### HashMap Operations

#### 1 `put(K key, V value)`

- Adds a key-value pair to the map.
- If the key **already exists**, it replaces the old value.
- **Returns `null`** if it's a new entry, otherwise returns the previous value.

```
HashMap<Integer, String> map = new HashMap<>();  
System.out.println(map.put(1, "A")); // null (new entry)  
System.out.println(map.put(2, "B")); // null (new entry)  
System.out.println(map.put(1, "C")); // "A" (old value replaced)  
System.out.println(map); // {1=C, 2=B}
```

## 2 putIfAbsent(K key, V value)

- Adds the key-value pair **only if the key is absent**.
- If the key already exists, it **retains the old value**.

```
map.putIfAbsent(1, "X"); // Won't update since 1 already exists  
System.out.println(map); // {1=C, 2=B}
```

## 3 putAll(Map<? extends K, ? extends V> m)

- Copies all entries from **one map to another**.

```
HashMap<Integer, String> map2 = new HashMap<>();  
map2.put(3, "D");  
map2.putAll(map);  
System.out.println(map2); // {3=D, 1=C, 2=B}
```

## 4 get(Object key)

- Retrieves the **value associated with the key**.
- Returns `null` if key **does not exist**.

```
System.out.println(map.get(2)); // B  
System.out.println(map.get(10)); // null
```

## 5 containsKey(Object key)

- Checks if a **key exists** in the map.
- **Time Complexity: O(1)** (hashing-based lookup).

```
System.out.println(map.containsKey(1)); // true  
System.out.println(map.containsKey(5)); // false
```

## 6 containsValue(Object value)

- Checks if a **value exists** in the map.
- **Time Complexity: O(n)** (iterates over all values).

```
System.out.println(map.containsValue("B")); // true
System.out.println(map.containsValue("X")); // false
```

## 7 remove(Object key)

- Removes the key-value pair **if the key exists**.
- Returns `null` if key is **not found**.

```
System.out.println(map.remove(2)); // B
System.out.println(map); // {1=C}
System.out.println(map.remove(10)); // null (key not found)
```

## How to Overcome Map Limitations?

Since `Map` does **not support direct iteration**, we **convert it to a collection view**.

### 1 Extracting Keys ( `keySet()` )

- Returns a **Set** of keys.

```
Set<Integer> keys = map.keySet();
System.out.println(keys); // [1]
```

### 2 Extracting Values ( `values()` )

- Returns a **Collection** of values.

```
Collection<String> values = map.values();
System.out.println(values); // [C]
```

### 3 Extracting Entries ( `entrySet()` )

- Returns a **Set of Map.Entry<K, V> pairs**.

```
Set<Map.Entry<Integer, String>> entries = map.entrySet();
for (Map.Entry<Integer, String> entry : entries) {
    System.out.println(entry.getKey() + " → " + entry.getValue());
}
// Output: 1 → C
```

## Summary: HashMap vs TreeMap vs LinkedHashMap

| Feature  | HashMap    | TreeMap                          | LinkedHashMap     |
|----------|------------|----------------------------------|-------------------|
| Ordering | ✗ No order | ✓ Sorted<br>(natural/comparator) | ✓ Insertion order |

|               |                                                             |                     |                    |
|---------------|-------------------------------------------------------------|---------------------|--------------------|
| Performance   | ✓ Fast (O(1) lookup)                                        | ✗ Slower (O(log n)) | ✓ Fast (O(1))      |
| Null Keys     | ✓ Allowed (only 1)                                          | ✗ Not allowed       | ✓ Allowed (only 1) |
| Thread Safety | ✗ No (use <code>ConcurrentHashMap</code> for thread safety) | ✗ No                | ✗ No               |

## 1. Generic Methods in Java

A **generic method** is a method that defines its own type parameter, independent of the class where it's declared.

### Key Points:

- It can exist in both **generic** and **non-generic** classes.
- The type parameter is defined **between the method modifiers and return type**.
- Example:

```
public static <T> List<T> asList(T... a) { }
```

### Example Usage:

```
List<Integer> intList = Arrays.asList(1, 2, 3, 4);
List<String> strList = Arrays.asList("A", "B", "C");
```

- `Arrays.asList()` is a **generic method in a non-generic class** (`Arrays`).

## 2. Generics - `?`, `extends`, `super`

### Hierarchy:

```
Emp ← Mgr ← SalesMgr
Emp ← Worker ← TempWorker
```

### Key Rules:

1. **Wildcards** (`?`) - Represents an unknown type.
2. **Upper Bound** (`extends`) - Restricts to a type and its subtypes.

```
List<? extends Emp> empList;
```

- Can hold **Emp, Mgr, SalesMgr, Worker, TempWorker**, but cannot add new elements.

3. **Lower Bound** (`super`) - Restricts to a type and its supertypes.

```
List<? super Mgr> mgrList;
```

- Can hold **Mgr**, **Emp**, **Object**, but not subtypes like `SalesMgr`.

### Example Scenarios:

```
ArrayList<?> l1 = new ArrayList<>();
ArrayList<Mgr> l2 = new ArrayList<>();
l1 = l2; //  Allowed
```

- Here, `?` represents an **unbounded wildcard**.

```
ArrayList<Mgr> mgrs = new ArrayList<>();
ArrayList<? extends Emp> empList = mgrs; //  Valid
ArrayList<? extends Object> objList = mgrs; //  Valid
```

- `mgrs` is a list of `Mgr`, which is a subtype of `Emp`, so it's assignable to `? extends Emp`.

## 3. Java 8 HashMap Changes

- In **Java 7 and earlier**, HashMap used a **linked list** to handle hash collisions.
- **Java 8** introduced **balanced trees** for buckets when collisions exceed a threshold.
- **Time complexity improvement:**
  - **Before Java 8:** Worst-case lookup is  **$O(n)$**  (traversing a linked list).
  - **After Java 8:** Worst-case lookup improves to  **$O(\log n)$**  (using a tree).

## 4. Hands-on Problems

### (a) Shuffle Elements of a List

```
Collections.shuffle(myList);
```

- Uses **unbounded wildcard** (`?`).

### (b) Reverse Elements of a List

```
Collections.reverse(myList);
```

### (c) Generic Method to Print Any List

```
public static <T> void printList(List<T> list) {
    for (T element : list) {
        System.out.println(element);
    }
}
```

- Can be used with `ArrayList<Integer>`, `LinkedList<String>`, `Vector<Double>`.

#### (d) Compute Total Salary of Employees

```
public static void computeTotalSalary(List<? extends Emp> employees) {
    double total = 0;
    for (Emp emp : employees) {
        total += emp.computeSalary();
    }
    System.out.println("Total Salary: " + total);
}
```

- Uses `? extends Emp` since we are **reading** from the list.

#### (e) Find Max from a List of Numbers

```
public static <T extends Comparable<T>> T findMax(List<T> list) {
    return Collections.max(list);
}
```

- Works for `Integer`, `Float`, `Double`, etc.

#### (f) Count Elements Greater Than a Given Element

```
public static <T extends Comparable<T>> int countGreaterThan(List<T> list, T elem) {
    int count = 0;
    for (T item : list) {
        if (item.compareTo(elem) > 0) {
            count++;
        }
    }
    return count;
}
```

- Can be used for `List<String>`, `List<Double>`, `List<Date>`, etc.

### 5. Understanding `copy()` in Collections

```
Collections.copy(destList, srcList);
```

#### Rule:

- **Source ( `src` ) should be a subtype of destination ( `dest` ).**
- Example:

```
List<Emp> empList = new ArrayList<>();
List<SalesMgr> salesMgrList = new ArrayList<>();
Collections.copy(empList, salesMgrList); //  Valid
```

#### Which are valid?

✓ SalesMgr → Emp

✓ SalesMgr → Mgr

✗ Emp → Mgr (wrong direction)

✗ Mgr → SalesMgr (wrong direction)

## 6. Difference Between Wildcard Bound and Type Parameter Bound

| Feature                 | Wildcard ( <code>?</code> )                                 | Type Parameter ( <code>T</code> )             |
|-------------------------|-------------------------------------------------------------|-----------------------------------------------|
| <b>Number of Bounds</b> | Only one ( <code>? extends</code> or <code>? super</code> ) | Multiple ( <code>T extends A &amp; B</code> ) |
| <b>Usage</b>            | Used for method <b>parameters</b>                           | Used for <b>entire class or method</b>        |
| <b>Flexibility</b>      | More flexible, allows different types                       | More restrictive                              |

### Example:

```
class Box<T extends Appendable & Flushable> { // Type parameter bound
    private T theObject;
    public Box(T arg) { theObject = arg; }
    public Box(Box<? extends T> box) { theObject = box.theObject; } // Wildcard bound
}
```

- `T extends Appendable & Flushable` → `T` must implement **both** interfaces.
- `? extends T` → Can be **T or any subclass**.

## 7. Consumer ( `super` ) vs. Producer ( `extends` )

| Situation                                          | Syntax                   |
|----------------------------------------------------|--------------------------|
| <b>Collection is a Producer</b> (Returning values) | <code>? extends T</code> |
| <b>Collection is a Consumer</b> (Accepting values) | <code>? super T</code>   |

### Example:

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (T item : src) {
        dest.add(item);
    }
}
```

- `src` is `? extends T` → It **produces** data.
- `dest` is `? super T` → It **consumes** data.

## Final Takeaways

1. **Generic Methods:** `<T> returnType methodName(T param)`
2. **Wildcards:**
  - `? extends T` → Use when **reading** data.

- `? super T` → Use when **writing** data.

### 3. Java 8 HashMap:

- Uses **balanced trees** instead of **linked lists** for handling hash collisions.
- Improves **worst-case complexity** from  $O(n)$  →  $O(\log n)$ .

### 4. Collections API: Methods like `copy()`, `sort()`, `shuffle()` use generics.

---

## 🌟 Why Generics Were Introduced in Java

### 1. Type Safety at Compile-Time

- Detect type mismatches early.
- Avoid `ClassCastException` at runtime.

### 2. Avoid Explicit Downcasting

- Before generics:

```
String s = (String) list.get(0); // Risky
```

- With generics:

```
String s = list.get(0); // Safe
```

### 3. Code Reusability & Flexibility

- Instead of writing separate classes for each type (`EmployeeList`, `CustomerList`, etc.), just use:

```
ArrayList<Employee> | ArrayList<Customer> | ArrayList<BankAccount>
```

## ⚙️ What is Type Erasure?

- Type erasure is a **compile-time process** where:
    - **Generic type parameters** are **replaced** with:
      - Their **first bound**, if bounded.
      - `Object`, if unbounded.
  - **Purpose:**
    - Maintain **backward compatibility** with legacy (pre-generic) Java code.
    - Avoid **runtime overhead** and **code bloat**.
- 

## 📌 Simple Rule

| During type erasure, the compiler:

- **Removes type parameters.**
  - **Replaces** them with:
    - **First bound** (if bounded).
    - **Object** (if unbounded).
- 

## Examples

### 1. Unbounded Type

```
public class Node<T> {  
    private T data;  
    ...  
}
```

 After erasure:

```
public class Node {  
    private Object data;  
    ...  
}
```

### 2. Multiple Generics

```
public class Pair<T, U> {  
    private T first;  
    private U second;  
    ...  
}
```

 After erasure:

```
public class Pair {  
    private Object first;  
    private Object second;  
    ...  
}
```

### 3. Bounded Type

```
public class Node<T extends Comparable> {  
    private T data;  
    ...  
}
```

 After erasure:

```
public class Node {
    private Comparable data;
    ...
}
```

## 4. Multiple Bounds

```
public class Node<T extends Comparable & Serializable> {
    private T data;
    ...
}
```

 After erasure:

```
public class Node {
    private Comparable data; // Only first bound retained
    ...
}
```

 **Note:**

- If the order is swapped:

```
<T extends Serializable & Comparable>
```

- Then `T` will be replaced with `Serializable`.

## Why Type Erasure Matters

| Benefit                         | Explanation                                                                   |
|---------------------------------|-------------------------------------------------------------------------------|
| ✓ <b>Backward Compatibility</b> | Allows generic code to work with old JVMs and non-generic legacy code.        |
| ✓ <b>No Runtime Overhead</b>    | Generics exist <b>only at compile time</b> → no additional classes or memory. |
| ✓ <b>No Code Bloat</b>          | Single version of bytecode → no duplication for each generic type used.       |

# Day 14

## ◆ What is Functional Programming?

Functional programming (FP) is a **declarative** programming style where you focus on **what** to do rather than **how** to do it. This is different from imperative programming (used in OOP), where you explicitly define each step to achieve a result.

### ◆ Key characteristics of FP:

- ✓ **Pure Functions** (No side effects, same output for same input)
- ✓ **Immutable Data** (No modification of existing data)

- ✓ **Higher-Order Functions** (Functions as first-class citizens)
- ✓ **Declarative Code** (Focus on results, not steps)
- ✓ **Lazy Evaluation** (Evaluation happens only when needed)

## ◆ Functional Programming vs Object-Oriented Programming (OOP)

| Feature             | OOP (Imperative)           | FP (Declarative)                  |
|---------------------|----------------------------|-----------------------------------|
| <b>Code Style</b>   | Step-by-step (how)         | Focuses on result (what)          |
| <b>State</b>        | Mutable (modifiable)       | Immutable (unchangeable)          |
| <b>Functions</b>    | Attached to objects        | Independent, first-class citizens |
| <b>Side Effects</b> | Common                     | Avoided                           |
| <b>Concurrency</b>  | Harder due to shared state | Easier due to immutability        |

## ◆ Higher-Order Functions (HOFs)

In Java, **functions are treated as first-class citizens**, meaning:

- ✓ Functions can be **assigned to variables**
- ✓ Functions can be **passed as arguments**
- ✓ Functions can be **returned from other functions**

### ◆ Example of HOF:

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<Integer, Integer> square = (x) → x * x;
        System.out.println(square.apply(5)); // Output: 25
    }
}
```

## ◆ Functional Interfaces in Java

A **functional interface** has exactly **one abstract method** (SAM - Single Abstract Method).

### ◆ Examples:

- ✓ `Runnable` (run method)
- ✓ `Comparator<T>` (compare method)
- ✓ `Consumer<T>` (accept method)
- ✓ `Predicate<T>` (test method)
- ✓ `Supplier<T>` (get method)
- ✓ `Function<T, R>` (apply method)

### ◆ Example: Creating a Functional Interface

```
@FunctionalInterface
interface MathOperation {
    int operation(int a, int b);
}
```

### ◆ Example: Using Lambda with Functional Interface

```
public class LambdaExample {
    public static void main(String[] args) {
        MathOperation addition = (a, b) → a + b;
        System.out.println(addition.operation(10, 5)); // Output: 15
    }
}
```

## ◆ Lambda Expressions

Lambda expressions are **anonymous functions** that provide a **concise way to write functions**.

### ◆ Syntax:

```
(parameters) → expression;
(parameters) → { statements; }
```

### ◆ Examples:

```
(a, b) → a + b;           // Addition function
name → name.toUpperCase(); // Convert string to uppercase
() → System.out.println("Hello"); // No parameter lambda
```

### ◆ Example in Java Streams:

```
List<String> names = Arrays.asList("Akshay", "Ravi", "Sita");
names.forEach(name → System.out.println(name));
```

## ◆ Method References

Method references provide a way to refer to **existing methods** without executing them.

### ◆ Syntax:

```
ClassName::methodName
```

### ◆ Example: Using Method Reference in Streams

```
List<String> names = Arrays.asList("Akshay", "Ravi", "Sita");
names.forEach(System.out::println);
```

## ◆ Java 8 Default & Static Methods in Interfaces

Java 8 introduced **default and static methods** in interfaces to add new functionality **without breaking old code**.

### ◆ Example of Default Method in Interface

```
interface Vehicle {
    default void horn() {
        System.out.println("Beep Beep!");
    }
}
class Car implements Vehicle {}
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.horn(); // Output: Beep Beep!
    }
}
```

### ◆ Example of Static Method in Interface

```
interface Utility {
    static int square(int x) {
        return x * x;
    }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(Utility.square(4)); // Output: 16
    }
}
```

## ◆ Java 8 Streams API

Streams allow **functional-style** operations on collections.

### ◆ Creating a Stream from List

```
List<String> names = Arrays.asList("Akshay", "Rahul", "Sita");
names.stream().forEach(System.out::println);
```

### ◆ Filtering a List using Stream

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.stream().filter(n → n % 2 == 0).forEach(System.out::println); // Output: 2, 4
```

### ◆ Summing All Elements using Stream

```
int sum = IntStream.range(1, 101).sum();  
System.out.println(sum); // Output: 5050
```

## ◆ Parallel Processing with Streams

```
List<String> names = Arrays.asList("Akshay", "Rahul", "Sita");  
names.parallelStream().forEach(System.out::println);
```

## ◆ Functional Programming Challenges for Practice

- 1 Use `forEach` to print all elements of a list
- 2 Remove all odd numbers from an `ArrayList`
- 3 Filter employees with a performance index `< 7`
- 4 Sort employees based on salary using a lambda comparator
- 5 Find the sum of all even numbers from 1 to 100 using Streams
- 6 Get all employees from a specific department who joined after a given date using Streams
- 7 Create a Supplier to generate random numbers
- 8 Convert a list of strings to uppercase using `map()`

## ◆ Summary

- ✓ **Functional Programming (FP)** makes Java more expressive, readable, and concise.
- ✓ **Lambdas & Functional Interfaces** allow passing behavior easily.
- ✓ **Streams API** enables **functional-style operations** on collections.
- ✓ **Method References** make code cleaner and reusable.
- ✓ **Default & Static Methods** in interfaces allow backward compatibility.
- ✓ **Parallel Streams** help leverage multi-core processors.



- **Stream Source**

- Streams can be created from Collections, Lists, Sets, ints, longs, doubles, arrays, lines of a file

- Stream operations are either intermediate or terminal.

- **Intermediate operations** such as filter, map or sort return a stream so we can chain multiple intermediate operations.

- **Terminal operations** such as forEach, collect or reduce are either void or return a non-stream result.

## 1 Default Methods in Interfaces (Java 8 Feature)

Java 8 introduced the `default` keyword, allowing interfaces to have method implementations without breaking existing code.

### Why `default` Methods?

1. **Backward Compatibility** – Ensures older interfaces can add new methods without breaking implementing classes.
2. **Avoid Code Duplication** – New functionality can be added without modifying all implementing classes.

### Example of a Default Method

```
interface Formula {  
    double calculate(double a); // Implicitly public and abstract  
  
    default double sqrt(double a, double b) {  
        return Math.sqrt(a + b); // Default method implementation  
    }  
}
```

### How Implementing Classes Handle Default Methods?

- ✓ **Inherit the Default Implementation** (if no override is needed)
- ✓ **Override if Needed**
- ✓ **Use** `InterfaceName.super.methodName(...)` **to call the default method explicitly**

## Handling Multiple Inheritance Issues

If a class implements multiple interfaces with the **same default method**, Java forces you to override it to resolve ambiguity.

### Example

```
interface A {
    default void show() { System.out.println("A's show method"); }
}
interface B {
    default void show() { System.out.println("B's show method"); }
}
class C implements A, B {
    // Must override show() to resolve ambiguity
    public void show() {
        A.super.show(); // Explicitly calling A's show method
    }
}
```

## 2 Static Methods in Interfaces

- ✓ Java 8 allows static methods inside interfaces.
- ✓ They **cannot be overridden** by implementing classes.
- ✓ They must be called using the **interface name**.

### Example

```
interface Utility {
    static int square(int x) {
        return x * x;
    }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(Utility.square(4)); // Output: 16
    }
}
```

## Key Points About Static Methods in Interfaces

- ✓ **Cannot be overridden** in implementing classes
- ✓ **Can be redeclared** in a subclass, but it will be a different method
- ✓ **Must be called using interface name**

## 3 Functional Interfaces

A **Functional Interface** has exactly **one abstract method** (SAM – Single Abstract Method).

✓ **@FunctionalInterface** annotation ensures compliance.

✓ Functional interfaces can be replaced with **lambda expressions**.

## Common Functional Interfaces in Java

| Interface                         | Method                               | Description                                     |
|-----------------------------------|--------------------------------------|-------------------------------------------------|
| <code>Runnable</code>             | <code>void run()</code>              | Used in Threads                                 |
| <code>Comparator&lt;T&gt;</code>  | <code>int compare(T o1, T o2)</code> | Sorting logic                                   |
| <code>Consumer&lt;T&gt;</code>    | <code>void accept(T t)</code>        | Takes input, returns nothing                    |
| <code>Predicate&lt;T&gt;</code>   | <code>boolean test(T t)</code>       | Returns <code>true</code> or <code>false</code> |
| <code>Supplier&lt;T&gt;</code>    | <code>T get()</code>                 | Returns a value, takes nothing                  |
| <code>Function&lt;T, R&gt;</code> | <code>R apply(T t)</code>            | Converts <code>T</code> to <code>R</code>       |

## Example of a Custom Functional Interface

```
@FunctionalInterface
interface MathOperation {
    int operation(int a, int b);
}
```

## Can the following interfaces be functional?

```
public interface A { double calc(int a,int b);} // ✓ Yes, single abstract method
public interface B extends A {} // ✓ Yes, since it inherits A's method
public interface C extends A { void show();} // ✗ No, more than one abstract method
public interface D {} // ✗ No abstract method
public interface E extends A {
    default void show(){ System.out.println("in show"); } // ✓ Yes, default method doesn't count
    static void test() { System.out.println("in test"); } // ✓ Yes
}
```

✓ **A, B, and E are valid functional interfaces**

✓ **C and D are not functional interfaces**

## 4 Lambda Expressions in Java 8

A **Lambda Expression** is an **anonymous function** used to simplify functional programming.

## Syntax of Lambda Expression

```
(parameters) → expression
(parameters) → { statements }
```

## Example: Using Lambda for Math Operations

```

@FunctionalInterface
interface MathOperation {
    double operate(double a, double b);
}

public class LambdaExample {
    public static void main(String[] args) {
        MathOperation add = (a, b) → a + b;
        MathOperation subtract = (a, b) → a - b;
        MathOperation multiply = (a, b) → a * b;
        MathOperation divide = (a, b) → a / b;

        System.out.println(add.operate(10, 5));    // Output: 15.0
        System.out.println(subtract.operate(10, 5)); // Output: 5.0
        System.out.println(multiply.operate(10, 5)); // Output: 50.0
        System.out.println(divide.operate(10, 5)); // Output: 2.0
    }
}

```

## Using Built-in Functional Interfaces with Lambda

```

import java.util.function.*;

public class LambdaBuiltInExample {
    public static void main(String[] args) {
        // Predicate: Checks if a number is even
        Predicate<Integer> isEven = n → n % 2 == 0;
        System.out.println(isEven.test(10)); // true

        // Consumer: Prints a message
        Consumer<String> printMessage = message → System.out.println(message);
        printMessage.accept("Hello, Akshay!");

        // Supplier: Returns a fixed value
        Supplier<Double> randomValue = () → Math.random();
        System.out.println(randomValue.get());

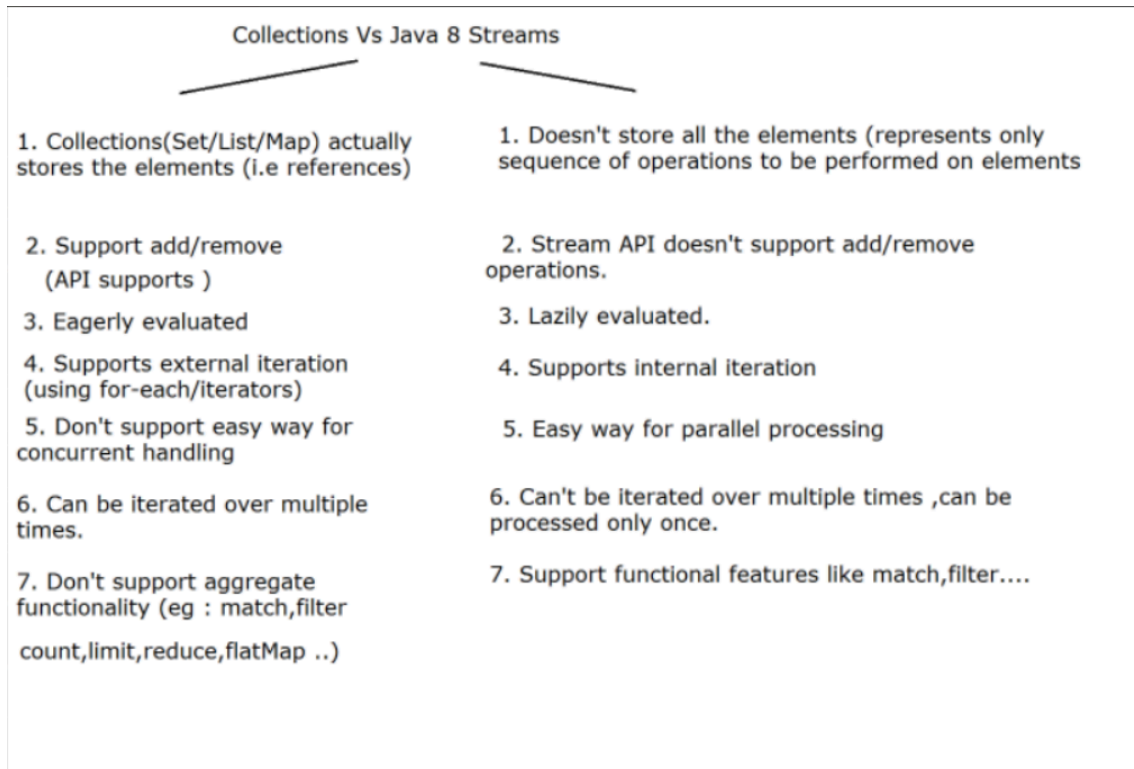
        // Function: Converts string to uppercase
        Function<String, String> toUpperCase = str → str.toUpperCase();
        System.out.println(toUpperCase.apply("akshay")); // Output: AKSHAY
    }
}

```

## Summary

### Java 8 Enhancements

1. **Default Methods** in interfaces allow new functionality without breaking old code.
2. **Static Methods** in interfaces provide utility methods without polluting implementing classes.
3. **Functional Interfaces** (Single Abstract Method) enable functional programming.
4. **Lambda Expressions** provide concise syntax for passing behavior as an argument.



## Day 15

### ◆ 1. Unbounded Wildcard `<?>`

- **Means:** Accepts any type.
- **Use case:** When you **only want to read** from a list, and you **don't care what type** it contains.

```
public void printList(List<?> list) {  
    for (Object item : list) {  
        System.out.println(item);  
    }  
}
```

📌 Think of it like a read-only book shelf – you can look at the books (read), but you can't add new ones.

### ◆ 2. Upper Bounded Wildcard `<? extends Type>`

- **Means:** Accepts a list of `Type` or any **subclass** of `Type`.

- **Use case:** When you want to **read** items that are at least of a specific type (like `Number` or its subclasses).

```
public double sum(List<? extends Number> numbers)
```

📌 Think of it like a shelf that only holds fruits ( `extends Fruit` ), so you can eat (read) any fruit, but you don't know which kind to add (no write).

### ◆ 3. Lower Bounded Wildcard `<? super Type>`

- **Means:** Accepts a list of `Type` or any **superclass** of `Type`.
- **Use case:** When you want to **add** elements of a specific type.

```
public void addNumbers(List<? super Integer> list)
```

📌 Think of it like a basket that holds anything above `Integer` in the hierarchy (like `Number`, `Object`). You can safely put `Integer`s in it (write), but when you take something out, you only know it's an `Object` (not safe to read as `Integer`).

### ✅ Quick Summary Table

| Wildcard                         | What it accepts          | You can... | You can't...        |
|----------------------------------|--------------------------|------------|---------------------|
| <code>&lt;?&gt;</code>           | Any type                 | Read       | Write               |
| <code>&lt;? extends T&gt;</code> | T or any subclass of T   | Read       | Write               |
| <code>&lt;? super T&gt;</code>   | T or any superclass of T | Write      | Read as Object only |

### 💡 Variance Explained (Fancy Terms in Simple Words)

| Term                  | Meaning in Java                                                        | Example                           |
|-----------------------|------------------------------------------------------------------------|-----------------------------------|
| <b>Invariance</b>     | <code>List&lt;String&gt;</code> $\neq$ <code>List&lt;Object&gt;</code> | You can't assign one to the other |
| <b>Covariance</b>     | <code>&lt;? extends T&gt;</code> – for <b>reading</b>                  | List of child types is allowed    |
| <b>Contravariance</b> | <code>&lt;? super T&gt;</code> – for <b>writing</b>                    | List of parent types is allowed   |

### 📌 PECS Rule (Easy way to remember)

PECS: Producer Extends, Consumer Super

- If the list **produces** items for you to use → use `extends`.
- If the list **consumes** items you add → use `super`.

### 🔧 Real Example: Copying Data

```
public static <T> void copy(List<? extends T> source, List<? super T> destination) {
    for (T item : source) {
        destination.add(item);
    }
}
```

```
}  
}
```

- `source` : **produces** data → `extends`
- `destination` : **consumes** data → `super`

## 🎯 Final Words

| Concept                          | Think like...                                |
|----------------------------------|----------------------------------------------|
| <code>&lt;? extends T&gt;</code> | Read-only container (e.g., fruits to eat)    |
| <code>&lt;? super T&gt;</code>   | Write-only container (e.g., bin to throw in) |
| <code>&lt;?&gt;</code>           | Safe-for-viewing only                        |

## ✅ Important Facts of Java 8 Streams

### ◆ 1. Streams are single-use (not reusable)

- Once a **terminal operation** is called, the stream is considered **consumed** and **closed**.
- Trying to use the same stream again will throw:

```
IllegalStateException: stream has already been operated upon or closed
```

🟢 **Collections** are reusable

🔴 **Streams** are **not** reusable after terminal operation

```
Stream<String> stream = list.stream();  
stream.forEach(System.out::println); // ✅ OK  
stream.forEach(System.out::println); // ❌ IllegalStateException
```

### ◆ 2. Streams follow vertical execution (pipeline-like)

- Execution of stream operations happens **vertically**, meaning:
  - Elements pass through the **entire chain of operations one-by-one**, **not** layer-by-layer.

📌 Example:

```
list.stream()  
  .filter(...)  
  .map(...)  
  .forEach(...);
```

✅ For each element, `filter → map → forEach` happens **in that order** before moving to next element.

### ◆ 3. Streams are lazily evaluated

- Intermediate operations like `filter()`, `map()` are **not executed immediately**.

- They are executed **only when a terminal operation** like `collect()` , `forEach()` , `count()` is called.

📌 Example:

```
list.stream()
    .filter(s → {
        System.out.println("Filtering " + s);
        return s.startsWith("A");
    }); // Nothing happens yet!

// Execution starts only when terminal op is called:
.count(); // Now filtering actually runs
```

## ✅ Summary Table

| Feature        | Streams                   | Collections |
|----------------|---------------------------|-------------|
| Reusability    | ❌ Not reusable            | ✅ Reusable  |
| Execution type | ✅ Vertical pipeline       | ❌ N/A       |
| Evaluation     | ✅ Lazy (intermediate ops) | ❌ Eager     |

Let me know if you want short MCQs or code-based questions on Java 8 Streams for practice!

## Day 16

## ✅ Java I/O – Overview

Java I/O deals with **transferring data between Java applications and external devices** like:

- **Files** (text/binary)
- **Console**
- **Memory**
- **Network sockets**
- **Pipes**

## Java I/O Streams (java.io package)

Java uses **streams** for I/O operations:

- `InputStream` / `Reader` – for reading data
- `OutputStream` / `Writer` – for writing data

| Type                    | Data Unit  | Base Classes               |
|-------------------------|------------|----------------------------|
| <b>Binary Stream</b>    | Bytes      | InputStream / OutputStream |
| <b>Character Stream</b> | Characters | Reader / Writer            |

## ◆ File-Based I/O

### ◆ Binary Stream Classes

| Class                         | Description           | Purpose       |
|-------------------------------|-----------------------|---------------|
| <code>FileInputStream</code>  | Reads bytes from file | Binary input  |
| <code>FileOutputStream</code> | Writes bytes to file  | Binary output |

#### Important Methods:

```
read(), read(byte[]), read(byte[], offset, len)
write(int), write(byte[]), write(byte[], offset, len)
```

### ◆ Character Stream Classes

| Class                   | Description                | Purpose     |
|-------------------------|----------------------------|-------------|
| <code>FileReader</code> | Reads characters from file | Char input  |
| <code>FileWriter</code> | Writes characters to file  | Char output |

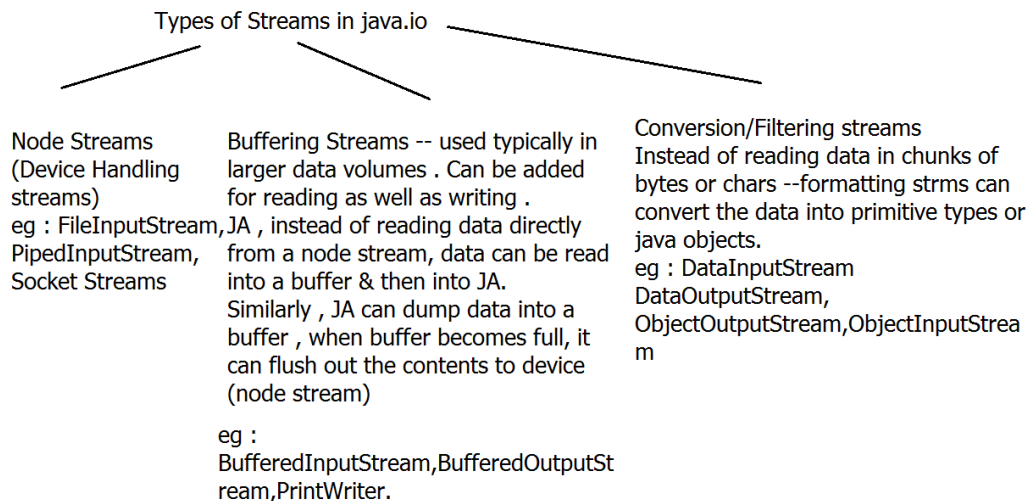
#### FileWriter Constructor

```
new FileWriter("abc.txt", true); // append = true
```

## ◆ Buffered I/O

Buffered streams improve performance by reducing disk access frequency.

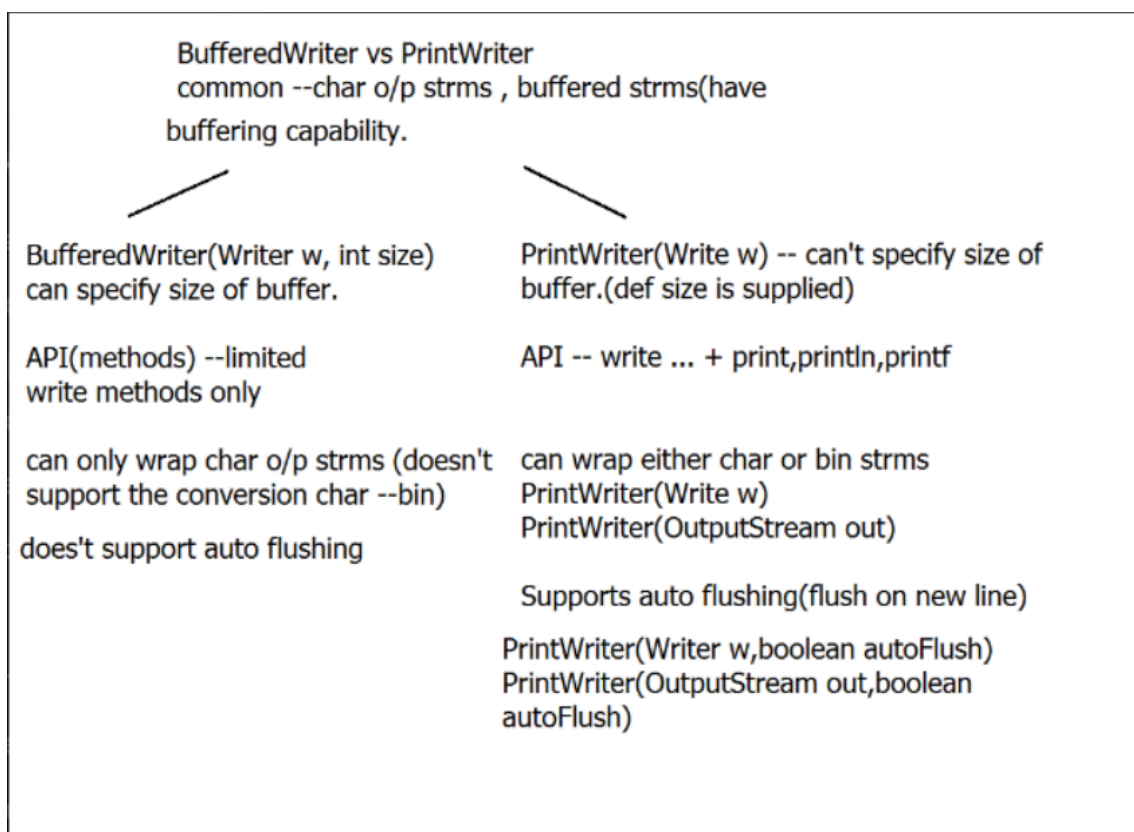
| Class                             | Purpose          | Supports                      |  |
|-----------------------------------|------------------|-------------------------------|--|
| <code>BufferedReader</code>       | Read lines/chars | <code>readLine()</code>       |  |
| <code>BufferedWriter</code>       | Write chars      | <code>write()</code>          |  |
| <code>BufferedInputStream</code>  | Read bytes       | <code>buffer + read()</code>  |  |
| <code>BufferedOutputStream</code> | Write bytes      | <code>buffer + write()</code> |  |
|                                   |                  |                               |  |



## ◆ **PrintWriter**

- Buffered char output stream with `print/println/printf` support.

```
PrintWriter pw = new PrintWriter(new FileWriter("output.txt", true), true);
pw.println("Hello"); // writes & flushes buffer automatically on newline
```



## ◆ **Reading Console Input**

| Technique                                                     | Notes                                              |
|---------------------------------------------------------------|----------------------------------------------------|
| <code>BufferedReader(new InputStreamReader(System.in))</code> | Buffered input from console                        |
| <code>Scanner(System.in)</code>                               | Provides parsing methods (nextInt, nextLine, etc.) |

📌 You can combine `BufferedReader` and `Scanner` :

```
Scanner sc = new Scanner(br.readLine()); // allows parsing line-by-line
```

## ◆ try-with-resources (Java 7+)

- Ensures automatic resource closing using `AutoCloseable` interface.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    // read data
} catch (IOException e) {
    e.printStackTrace();
}
```

## ◆ java.io.File Class

- Represents a path to file/folder
- Doesn't follow stream hierarchy (extends `Object`)
- Useful methods:

```
exists(), isFile(), canRead(), mkdir(), delete(), length(), lastModified()
```

## ◆ Text File Copy (Buffered Way)

```
BufferedReader br = new BufferedReader(new FileReader("source.txt"));
PrintWriter pw = new PrintWriter(new FileWriter("dest.txt"));
String line;
while ((line = br.readLine()) != null) {
    pw.println(line);
}
br.close();
pw.close();
```

## ◆ Binary Data Streams

| Class                         | Purpose                          |
|-------------------------------|----------------------------------|
| <code>DataOutputStream</code> | Write primitive types to stream  |
| <code>DataInputStream</code>  | Read primitive types from stream |

```
DataOutputStream dos = new DataOutputStream(new FileOutputStream("data.bin"));
dos.writeInt(10);
dos.writeUTF("Hello");
```

```
DataInputStream dis = new DataInputStream(new FileInputStream("data.bin"));
int num = dis.readInt();
String text = dis.readUTF();
```

## ◆ Serialization / Deserialization

| Class                           | Purpose                                |
|---------------------------------|----------------------------------------|
| <code>ObjectOutputStream</code> | Write objects to stream (serialize)    |
| <code>ObjectInputStream</code>  | Read objects from stream (deserialize) |

### Rules:

- Classes must implement `Serializable`
- Only **non-transient**, **non-static** fields are serialized.

```
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("emp.ser"));
oos.writeObject(emp); // emp must be Serializable
```

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("emp.ser"));
Employee emp = (Employee) ois.readObject();
```

## ◆ Collections + I/O

### Example: Item Inventory

```
class Item implements Serializable {
    String code, desc, category;
    int quantity;
    double price;
    LocalDate shipmentDate;
    // constructor, toString
}
```

### Store items:

- Sort `HashMap<String, Item>` based on `code` or `shipmentDate`
- Write line-by-line to text file using `PrintWriter`

### Restore items:

- Read lines, parse, and repopulate `HashMap`

## ◆ Advanced Scenario

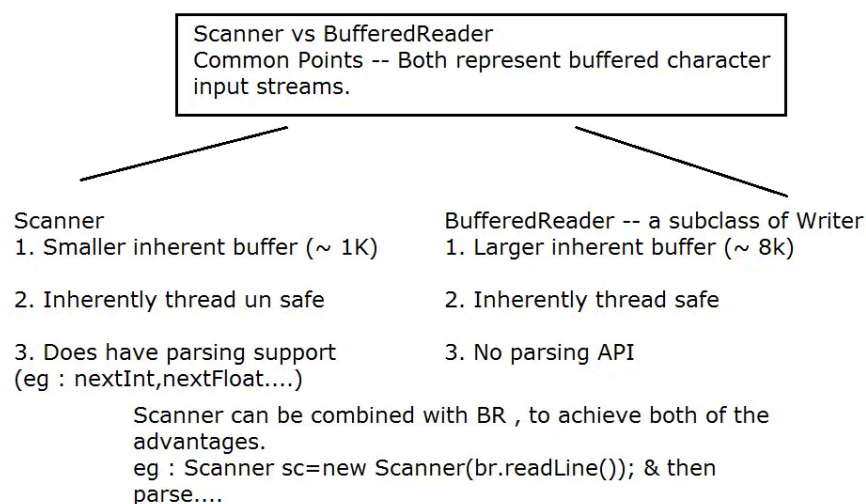
### Customer Data + Item Purchase List

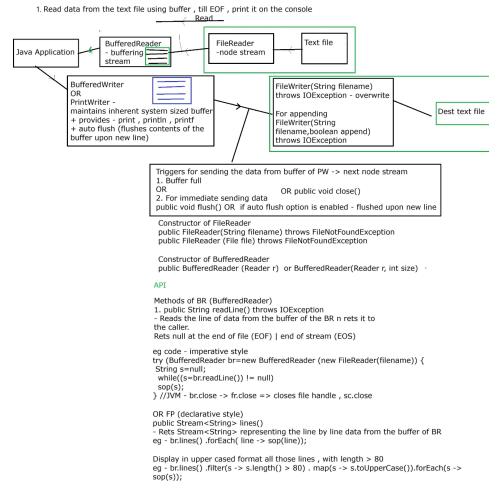
```
class CustomerPurchase implements Serializable {  
    int purchaseNo;  
    Customer customer;  
    ArrayList<Item> items;  
    LocalDate creationDate;  
}
```

- Store to file using `ObjectOutputStream`
- Restore from file using `ObjectInputStream`

## ◆ Java I/O Summary Table

| Goal                   | Classes Used                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------|
| Read/write text        | <code>FileReader</code> , <code>FileWriter</code> , <code>BufferedReader</code> , <code>PrintWriter</code> |
| Read/write binary      | <code>FileInputStream</code> , <code>FileOutputStream</code>                                               |
| Buffered binary        | <code>BufferedInputStream</code> , <code>BufferedOutputStream</code>                                       |
| Primitive type I/O     | <code>DataInputStream</code> , <code>DataOutputStream</code>                                               |
| Object I/O             | <code>ObjectOutputStream</code> , <code>ObjectInputStream</code>                                           |
| Read console input     | <code>BufferedReader</code> , <code>Scanner</code>                                                         |
| Auto-closing resources | <code>try-with-resources</code> , <code>AutoCloseable</code>                                               |
| File/folder operations | <code>java.io.File</code>                                                                                  |





## Day 17

### ✓ 1. java.io.File Class

- Represents an **abstract path** to a file or folder.
- **Doesn't handle I/O** directly (not a stream).
- Used to check file properties before attaching a stream.

```
File f = new File("abc.txt");
if (f.exists() && f.isFile() && f.canRead()) {
    // attach FileReader or FileInputStream
}
```

### ✓ 2. Storing Data in Machine-Independent Format

#### ◆ Using **DataOutputStream**

- Converts **primitive types / String** → **binary**
- **Automatically handles encoding**
- Ideal for writing structured data to binary files

```
DataOutputStream dos = new DataOutputStream(
    new BufferedOutputStream(new FileOutputStream("product.bin")));
dos.writeUTF("P101");
dos.writeDouble(1500.75);
dos.writeInt(20);
dos.close();
```

#### ◆ Using **DataInputStream**

- Converts **binary** → **primitive types / String**

```
DataInputStream dis = new DataInputStream(  
    new BufferedInputStream(new FileInputStream("product.bin")));  
String code = dis.readUTF();  
double price = dis.readDouble();  
int qty = dis.readInt();  
dis.close();
```

### ✓ 3. Why Object Streams?

Without object streams:

- Developer must **manually write and read each field** using DataOutputStream/DataInputStream.
- Tedious and error-prone.

With object streams:

- Automatically serialize the entire object (including inner object graph) to a binary file.

### ✓ 4. Serialization with **ObjectOutputStream**

- Converts **entire object** → **binary stream**
- Stores **object state** (non-static, non-transient fields)

```
ObjectOutputStream oos = new ObjectOutputStream(  
    new FileOutputStream("emp.ser"));  
oos.writeObject(emp); // emp must be Serializable  
oos.close();
```

#### 🟡 Requirements:

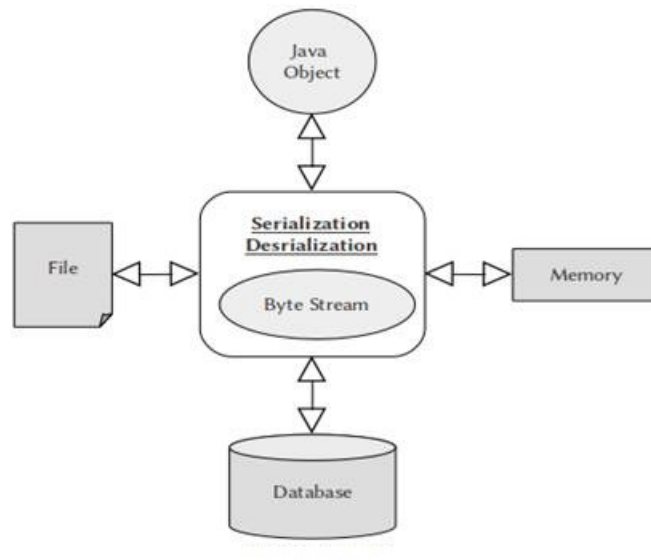
- Class must implement: `java.io.Serializable` (marker interface)
- All non-transient fields and their objects must also be serializable

### ✓ 5. Deserialization with **ObjectInputStream**

```
ObjectInputStream ois = new ObjectInputStream(  
    new FileInputStream("emp.ser"));  
Employee emp = (Employee) ois.readObject();  
ois.close();
```

#### 📌 Exceptions:

- `ClassNotFoundException` – if class definition not found at runtime
- `InvalidClassException` – mismatch in `serialVersionUID`
- `IOException` – general I/O issue



## ✓ 6. `serialVersionUID`

| Concept                       | Description                                                                    |
|-------------------------------|--------------------------------------------------------------------------------|
| <code>serialVersionUID</code> | A unique version ID for the class                                              |
| Purpose                       | Ensures version compatibility during de-serialization                          |
| Recommended Practice          | Manually define: <code>private static final long serialVersionUID = 1L;</code> |

If not manually declared, JVM auto-generates it (may cause issues across different JVM versions or after class changes).

## ✓ 7. `transient` Keyword

- Used to **exclude a field** from serialization.
- During deserialization, **transient fields are set to default** (e.g., 0 for int, null for objects)

```
transient String password;
```

## ✓ 8. Cloning in Java

### ◆ Shallow Copy (default)

- Uses `Object.clone()` method

- Requires class to implement `java.lang.Cloneable` (marker interface)
- Override `clone()` and widen visibility to `public`

```
@Override
public Emp clone() throws CloneNotSupportedException {
    return (Emp) super.clone(); // Shallow copy
}
```

## ◆ Deep Copy

- You must **manually clone** all mutable or reference-type fields

```
@Override
public Emp clone() throws CloneNotSupportedException {
    Emp cloned = (Emp) super.clone();
    cloned.adr = adr.clone(); // assuming Address also supports clone
    return cloned;
}
```

## ✓ Summary Table

| Task                             | Stream / Class                                       | Notes                                                  |
|----------------------------------|------------------------------------------------------|--------------------------------------------------------|
| Write primitive to binary        | <code>DataOutputStream</code>                        | <code>writeInt()</code> , <code>writeUTF()</code> etc. |
| Read primitive from binary       | <code>DataInputStream</code>                         | <code>readInt()</code> , <code>readUTF()</code> etc.   |
| Write object to binary           | <code>ObjectOutputStream</code>                      | Requires <code>Serializable</code>                     |
| Read object from binary          | <code>ObjectInputStream</code>                       | Returns <code>Object</code> , needs casting            |
| Exclude field from serialization | <code>transient</code> keyword                       | Skips field during serialization                       |
| Maintain version compatibility   | <code>serialVersionUID</code>                        | Define manually                                        |
| Clone an object                  | <code>clone()</code> method + <code>Cloneable</code> | Use deep copy if object has graph                      |

## ✓ Use Case Example

```
class Product implements Serializable {
    private static final long serialVersionUID = 1L;
    String code;
    String name;
    double price;
    transient int stock; // won't be serialized
}
```

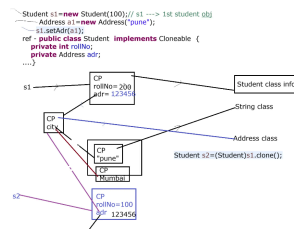
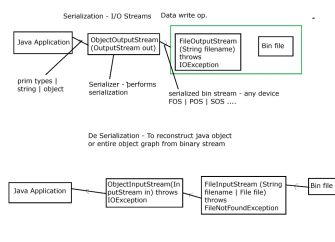
### Storing object:

```
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("product.ser"));
oos.writeObject(product);
```

```
oos.close();
```

## Restoring object:

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("product.ser"));
Product p = (Product) ois.readObject();
ois.close();
```



## ✓ What's Left to Explain (Unexplained Earlier)

### ◆ 1. Why constructor of Serializable class is *not* called during deserialization?

#### Reason:

During deserialization, Java needs to restore the exact **saved state** of an object **as it was**—so instead of calling the constructor (which may initialize or alter fields), it:

- Allocates memory using low-level reflection (like `Class.newInstance()` or similar),
- Then directly injects field values from the serialized binary stream.

This ensures the object's state remains exactly the same as when it was serialized, rather than getting reset via constructor logic.

### ◆ 2. Deserialization Behavior in Inheritance Hierarchy

#### ✓ If *all* superclasses are serializable:

- JVM will not invoke *any* constructor.
- Object state will be restored directly from the stream.

#### ✗ If superclass is not serializable:

- JVM **must call its no-arg constructor** to initialize that part of the object.
- If it **does not exist**, it throws:

`java.io.InvalidClassException: no valid constructor`

🔄 This is because the state of a non-serializable superclass must be initialized the usual way — by constructor (since it wasn't serialized).

### ◆ 3. What exactly happens during Deserialization?

Here's the **step-by-step process**:

1. JVM reads the **class metadata** (name, `serialVersionUID`, field info).
2. **Checks compatibility**:
  - If `serialVersionUID` of class  $\neq$  UID from stream  $\rightarrow$  `InvalidClassException`
3. JVM finds the **first non-serializable superclass** and **calls its no-arg constructor**.
4. JVM uses **reflection to allocate memory** for the serializable parts (skipping constructor).
5. JVM reads field values from the stream and **sets them**.
6. `readObject()` method (default or overridden) is invoked (if present).
7. Object is returned — fully reconstructed.

### ◆ 4. Serialization Format (Structure of Binary File)

Serialized stream is not just field values — it contains:

| Section           | Description                                                              |
|-------------------|--------------------------------------------------------------------------|
| Magic Number      | A constant value <code>0xACED</code> (identifies Java serialized stream) |
| Version           | Stream format version                                                    |
| Class Description | Class name, UID, fields, etc.                                            |
| Object Data       | Field values (state)                                                     |
| Object Graph      | Includes all referenced serializable objects                             |
| End Marker        | Marks the end of object                                                  |

### ◆ 5. Limitations of Java Serialization

1. **Java only** – Not readable across different languages.
2. **Version-sensitive** – Any incompatible change  $\rightarrow$  breaks deserialization.
3. **Security risk** – Attacker can send malicious serialized data.
4. **Performance overhead** – Reflection, metadata, etc.
5. **Hidden logic** – Black box format, not human-readable.
6. **Deep object graph** – If one reference is non-serializable  $\rightarrow$  whole object fails.

### ◆ 6. Serialization vs Externalization

| Feature     | Serializable                              | Externalizable                                                           |
|-------------|-------------------------------------------|--------------------------------------------------------------------------|
| Control     | Automatic                                 | Manual                                                                   |
| Methods     | None (marker i/f)                         | Must override <code>writeExternal()</code> , <code>readExternal()</code> |
| Constructor | Not required                              | <b>Public no-arg constructor</b> required                                |
| Performance | Slower, more flexible                     | Faster, more compact                                                     |
| Use-case    | Simpler objects, no custom control needed | Full control over serialization, e.g. exclude large graphs, reduce size  |

## ◆ 7. Using Serialization for Deep Cloning

**Clone** only gives a **shallow copy** unless deep copy is implemented manually.

But you can easily get a deep copy via serialization:

```
public static Object deepClone(Object obj) throws Exception {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    ObjectOutputStream oos = new ObjectOutputStream(baos);
    oos.writeObject(obj);

    ByteArrayInputStream bais = new ByteArrayInputStream(baos.toByteArray());
    ObjectInputStream ois = new ObjectInputStream(bais);
    return ois.readObject();
}
```

✅ This creates a **true deep copy**, assuming all referenced objects are also serializable.

## ◆ 8. Final static vs non-final static during deserialization

| Type of Field             | Behavior                                                                |
|---------------------------|-------------------------------------------------------------------------|
| <code>static</code>       | Not serialized at all. Initialized from class definition at runtime     |
| <code>final static</code> | Value still available (from class definition, not stream)               |
| <code>transient</code>    | Skipped during serialization. Default values set during deserialization |

## ◆ 9. Compatible vs Incompatible Class Changes

| Compatible Changes ✅             | Incompatible Changes ❌                                                                                        |
|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| Adding new fields                | Removing fields                                                                                               |
| Adding new classes/interfaces    | Changing field type                                                                                           |
| Changing access modifiers        | Changing <code>non-static</code> → <code>static</code> or <code>non-transient</code> → <code>transient</code> |
| Adding <code>Serializable</code> | Renaming or moving class                                                                                      |

➡ For incompatible changes, always increment `serialVersionUID` .

## ◆ 10. How to Handle Non-Serializable References Inside Serializable Class

✅ Use `transient` :

```
class Product implements Serializable {  
    private String name;  
    private transient Connection dbConnection; // Won't be serialized  
}
```

➡ On deserialization, `dbConnection` will be `null`.

## ✅ Remaining Reflection Topics (Not Explained Earlier)

### ◆ 1. How to create an instance using the default constructor?

You can use:

```
T obj = clazz.newInstance();
```

#### ◆ Method:

```
public T newInstance() throws InstantiationException, IllegalAccessException
```

- It **creates a new instance using the no-arg (default) constructor**.
- Works only if:
  - Class has a public no-arg constructor.
  - Class is not abstract or interface.

⚠️ Deprecated since Java 9 — preferred approach:  
`clazz.getDeclaredConstructor().newInstance();`

### ◆ 2. How to create instance using parameterized constructor?

#### ✅ Step-by-step:

```
// 1. Get Constructor object  
Constructor<MyClass> cons = MyClass.class.getConstructor(String.class, int.class);  
  
// 2. Create object using constructor  
MyClass obj = cons.newInstance("Akshay", 25);
```

#### ◆ Class API:

```
public Constructor<T> getConstructor(Class<?>... parameterTypes)
```

#### ◆ Constructor API:

```
public T newInstance(Object... initargs)
```

### ◆ 3. How to invoke a private method dynamically?

Let's say a class `Student` has:

```
private void displayMarks(String subject) {  
    System.out.println("Subject: " + subject);  
}
```

#### ✓ Steps to invoke:

```
// 1. Get Class object  
Class<?> cls = Student.class;  
  
// 2. Get Method object (private)  
Method m = cls.getDeclaredMethod("displayMarks", String.class);  
  
// 3. Make it accessible  
m.setAccessible(true);  
  
// 4. Invoke the method  
Student s = new Student();  
m.invoke(s, "Java");
```

#### ◆ Method Class APIs:

- `Method getDeclaredMethod(String name, Class<?>... parameterTypes)`
- `void setAccessible(boolean flag)` — bypasses Java access checks
- `Object invoke(Object obj, Object... args)`

! If method is static, you can pass null as the object in `invoke()`.

#### ! Exception Handling Summary for Reflection:

| API                           | Exception(s)                                                                                                         |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>Class.forName()</code>  | <code>ClassNotFoundException</code>                                                                                  |
| <code>getConstructor()</code> | <code>NoSuchMethodException</code> , <code>SecurityException</code>                                                  |
| <code>newInstance()</code>    | <code>InstantiationException</code> , <code>IllegalAccessException</code> , <code>InvocationTargetException</code>   |
| <code>invoke()</code>         | <code>IllegalAccessException</code> , <code>IllegalArgumentException</code> , <code>InvocationTargetException</code> |

#### ◆ Bonus: When do we use `.TYPE` ?

For **primitive types**, Java provides `.TYPE` field in wrapper classes:

```
System.out.println(Integer.TYPE); // prints int
System.out.println(Boolean.TYPE); // prints boolean
```

This is helpful when calling constructors or methods reflectively with primitive parameter types.

## Day 18

### ✓ Java Threading – Full Overview

#### ◆ 1. Thread Creation Approaches

##### A Extending `Thread` class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Running in thread: " + Thread.currentThread().getName());
    }
}
```

Usage:

```
Thread t1 = new MyThread();
t1.start();
```

##### B Implementing `Runnable` (Recommended in real apps)

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Running in: " + Thread.currentThread().getName());
    }
}
```

Usage:

```
Runnable r = new MyTask();
Thread t2 = new Thread(r, "Worker-1");
t2.start();
```

##### AB Using Lambda (Since Java 8)

```
Thread t3 = new Thread(() → {
    System.out.println("Running with Lambda in: " + Thread.currentThread().getName());
}, "Lambda-Thread");
```

```
t3.start();
```

## ◆ 2. Important Thread Class APIs

| Method                                          | Description                                                                             |
|-------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>start()</code>                            | Launches new thread (calls internal <code>run()</code> on a new call stack)             |
| <code>run()</code>                              | Entry point logic (overridden from <code>Runnable</code> )                              |
| <code>yield()</code>                            | Suggests CPU to switch to another thread (cooperative multitasking)                     |
| <code>sleep(ms)</code>                          | Puts thread to sleep (blocked state)                                                    |
| <code>interrupt()</code>                        | Wakes a thread blocked on <code>sleep</code> , <code>join</code> , or <code>wait</code> |
| <code>join()</code>                             | Waits for another thread to finish                                                      |
| <code>currentThread()</code>                    | Gets current thread instance                                                            |
| <code>setName()</code> , <code>getName()</code> | Naming a thread                                                                         |
| <code>setPriority()</code>                      | Sets thread priority (1–10)                                                             |
| <code>toString()</code>                         | Prints thread info                                                                      |

## ◆ 3. Thread Lifecycle States

```
NEW → RUNNABLE → RUNNING → TERMINATED
    ↘
    BLOCKED
    ↘
    WAITING / TIMED_WAITING
```

## ◆ 4. Thread Priorities

| Constant                   | Value |
|----------------------------|-------|
| <code>MIN_PRIORITY</code>  | 1     |
| <code>NORM_PRIORITY</code> | 5     |
| <code>MAX_PRIORITY</code>  | 10    |

Priority only gives a **hint** to the scheduler. Not reliable for control.

## ◆ 5. `join()` Usage — Waiting for Threads to Finish

```
Thread t1 = new Thread() → {
    // Some task
};
t1.start();

t1.join(); // Current thread (e.g., main) waits for t1 to finish
```

Variants:

- `join()` – wait indefinitely
- `join(long ms)` – wait up to given ms

## ◆ 6. `interrupt()` and Handling `InterruptedException`

```
Thread t = new Thread(() → {
    try {
        Thread.sleep(5000);
    } catch (InterruptedException e) {
        System.out.println("Thread was interrupted!");
    }
});

t.start();
t.interrupt(); // Wakes up the sleeping thread
```

## ◆ Inter-Thread Communication (wait/notify)

| Based on Object class monitor methods. Works only in synchronized context.

### ◆ `wait()`

- Thread **releases the lock** and enters waiting state.
- Must be called from synchronized context.
- Throws:
  - `InterruptedException` – if interrupted
  - `IllegalMonitorStateException` – if thread is not owner of monitor

### ◆ `notify()`

- Wakes up **one** waiting thread from the monitor pool.

### ◆ `notifyAll()`

- Wakes up **all** waiting threads.

## ◆ Inter-thread Communication Example

```
class Shared {
    void printMsg() throws InterruptedException {
        synchronized (this) {
            System.out.println("Thread " + Thread.currentThread().getName() + " is waiting...");
            wait();
            System.out.println("Thread " + Thread.currentThread().getName() + " is resumed!");
        }
    }
}
```

```

    }

    void notifyMsg() {
        synchronized (this) {
            notify(); // OR notifyAll();
            System.out.println("Notification sent by " + Thread.currentThread().getName());
        }
    }
}

```

## ◆ Summary of Exceptions

| Method                            | Exception                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------|
| <code>start()</code>              | <code>IllegalThreadStateException</code>                                      |
| <code>sleep()</code>              | <code>InterruptedException</code>                                             |
| <code>join()</code>               | <code>InterruptedException</code>                                             |
| <code>wait()</code>               | <code>InterruptedException</code> , <code>IllegalMonitorStateException</code> |
| <code>notify()/notifyAll()</code> | <code>IllegalMonitorStateException</code>                                     |

## ◆ Recommended Practices

- Use `Runnable` with `Thread` (instead of extending `Thread` )
- Avoid using `yield()` and `setPriority()` for logic control
- Use `ExecutorService` in production for thread pooling
- Always handle `InterruptedException` properly
- Use `wait/notify` only when necessary – prefer `java.util.concurrent` tools like `CountDownLatch` , `Semaphore` , `BlockingQueue` , etc.

## ✓ New Key Points to Understand (Not covered earlier)

### ◆ 1. Static vs Non-static synchronized methods — Locking Difference

| Method Type             | Lock is on          | Example                      |
|-------------------------|---------------------|------------------------------|
| Non-static synchronized | Object instance     | <code>this</code>            |
| Static synchronized     | Class-level monitor | <code>ClassName.class</code> |

#### 📌 Example:

```

class Printer {
    public synchronized void printNonStatic() {
        // Lock is on this
    }
}

```

```
public static synchronized void printStatic() {
    // Lock is on Printer.class
}
}
```

#### Explanation:

- `printNonStatic()` acquires **object-level lock**: if two threads call this on the **same instance**, one waits.
- `printStatic()` acquires **class-level lock**: all threads calling this method will compete for a single lock **regardless of instance**.

#### ✅ Key Point:

Static and non-static synchronized methods do not interfere with each other.

### ◆ 2. Thread calling synchronized method on two different instances

#### 📌 Example:

```
Printer p1 = new Printer();
Printer p2 = new Printer();

Thread t1 = new Thread(() -> p1.print());
Thread t2 = new Thread(() -> p2.print());
```

#### Explanation:

- These two threads **won't block each other**, because each is locking on a different object ( `p1` and `p2` ).
- So, **object-level synchronization is per-instance**.

### ◆ 3. Reentrant Locks (same thread acquiring same lock multiple times)

Java's intrinsic locking is reentrant, meaning if a thread already holds a lock, it can re-enter any synchronized block/method protected by the same lock.

#### 📌 Example:

```
class A {
    synchronized void method1() {
        method2(); // same thread can call this
    }

    synchronized void method2() {
        System.out.println("method2");
    }
}
```

```
}  
}
```

#### Explanation:

- Thread holding lock on `this` can safely re-enter `method2` without deadlock.

### ◆ 4. Nested synchronized blocks

Sometimes, you need to **lock on multiple resources**.

#### 📌 Example:

```
synchronized (obj1) {  
    synchronized (obj2) {  
        // critical section requiring both locks  
    }  
}
```

**Be careful** — nested locking like this can lead to **deadlock** if lock ordering isn't consistent across threads.

### ◆ 5. Synchronization vs Performance

| Synchronization is expensive in terms of performance.

Therefore:

- **Don't synchronize entire methods unless necessary**
- Prefer **synchronized blocks** for smaller, critical sections only
- Use `java.util.concurrent` alternatives (e.g. `ConcurrentHashMap`, `ReentrantLock`) when possible

### ◆ 6. IllegalMonitorStateException

#### Occurs when:

You call `wait()`, `notify()`, or `notifyAll()` **outside** a synchronized context.

#### 📌 Example (wrong code):

```
List<Integer> list = new ArrayList<>();  
list.wait(); // ❌ IllegalMonitorStateException
```

#### Fix:

```
synchronized (list) {  
    list.wait(); // ✅  
}
```

## ◆ 7. Thread never releases the lock on sleep, yield, notify, or join

### ✓ Clarification:

- These **do NOT release the monitor (lock)**.
- Thread is just **temporarily suspended**, but still owns the lock.

## ◆ 8. When to Use Synchronized Block vs Synchronized Method

| Criteria                         | Recommendation                     |
|----------------------------------|------------------------------------|
| Small part of method is critical | Use synchronized block             |
| Entire method is critical        | Use synchronized method            |
| Working with thread-unsafe APIs  | Use synchronized block selectively |

## ◆ 9. Why we avoid synchronizing on String or wrapper classes

synchronized ("LOCK") // ❌ risky

- String literals are shared in the string pool, so other parts of the code (even in other libraries) might lock on the same object — leading to **unexpected blocking or deadlocks**.
- Better to synchronize on **private final Object lock = new Object();**

## ◆ 10. Real-world Example: Synchronizing access to shared collection

```
List<String> list = new ArrayList<>();

synchronized (list) {
    list.add("Java");
}
```

OR

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
```

### ✓ Conclusion

The newly covered concepts in this explanation:

- Static vs non-static synchronized difference
- Reentrant locking
- Nested synchronized blocks
- Performance trade-offs
- IllegalMonitorStateException handling
- Lock not released on sleep/join/yield

- String sync risk
- Practical synchronized block usage

Let me know if you'd like:

- **Race condition code demo**
- **Deadlock creation and resolution**
- MCQs or interview questions from this topic

Would you like to move on to `ReentrantLock`, `volatile`, `atomic variables`, or the **ExecutorService framework** next?

## Day 19

Great, Akshay! Here's a quick revision summary based on your notes:

### ✓ Thread Lifecycle & State Transitions

- **NEW → RUNNABLE → RUNNING → TERMINATED**
- Transitions via:
  - `start()` → RUNNABLE
  - `sleep()`, `wait()` → TIMED\_WAITING/WAITING
  - Completion or `stop()` → TERMINATED
  - `join()` makes calling thread WAITING
  - `notify()` / `notifyAll()` moves WAITING → RUNNABLE

### ✓ Thread API Highlights

- `start()`, `run()`, `join()`, `sleep()`, `yield()`, `interrupt()`, `setName()`, `setPriority()`, `currentThread()`
- `wait()`, `notify()`, `notifyAll()` (from `Object` class, used for inter-thread communication)

## ✓ 1. Inter-Thread Communication (ITC) – wait/notify Approach

Even with `synchronized`, problems like **data loss or redundancy** can occur in Producer-Consumer problems.

### ◆ Why?

Because synchronization **only controls access**, not **communication**.

### ◆ Solution: Use ITC via `wait()` and `notify()` :

### ✓ Shared Resource Example:

```
class SharedData {
    private Emp emp;
    private boolean dataReady = false;
```

```

public synchronized void write(Emp e) throws InterruptedException {
    while (!dataReady) wait(); // Wait if previous data is not yet read
    emp = e;
    dataReady = true;
    notify(); // Wake up the consumer
}

public synchronized Emp read() throws InterruptedException {
    while (!dataReady) wait(); // Wait if no data is written
    dataReady = false;
    notify(); // Wake up the producer
    return emp;
}
}

```

## ✓ 2. Problem with `Thread.sleep()` in ITC

### ◆ Why `sleep()` is dangerous in multithreaded communication?

- It **does not release the lock**.
- Causes **cyclic dependency** (deadlock potential).
- Thread **sleeps inside the monitor** ⇒ blocks other threads forever.

### ◆ Better Alternative:

Use `wait()` → releases the monitor + blocks safely until `notify()` is called.

## ✓ 3. `volatile` Keyword in Java

### ◆ Purpose:

Ensures **visibility** of changes to variables across threads. Avoids stale values due to thread-local caching.

```
private volatile boolean flag = true;
```

### ◆ When to use?

- In simple flags or status-checking where **only visibility** matters (no atomicity).
- Not suitable where **compound actions** (like increment) are needed. Use `AtomicInteger` there.

## ✓ 4. Daemon Threads vs User Threads

| Feature      | User Thread                | Daemon Thread                               |
|--------------|----------------------------|---------------------------------------------|
| Priority     | Higher (application logic) | Lower (background tasks)                    |
| JVM Shutdown | JVM waits for user threads | JVM <b>does NOT</b> wait for daemon threads |

| Feature | User Thread                 | Daemon Thread                    |
|---------|-----------------------------|----------------------------------|
| Example | Main thread, business logic | Garbage Collector, Timer threads |

#### ◆ Set daemon status before start:

```
Thread t = new Thread(...);
t.setDaemon(true);
t.start();
```

## ✓ 5. Random Number Generation – `java.util.Random`

#### ◆ Usage:

```
Random rand = new Random();
int n = rand.nextInt();    // Any int
int n2 = rand.nextInt(100); // 0 to 99
```

✓ Thread-safe alternative in concurrent apps: `ThreadLocalRandom` or `SecureRandom` for cryptographic needs.

## ✓ 6. Method Reference in Java (Java 8+)

Shortcut for writing lambda expressions when method already exists.

#### ◆ Syntax Types:

| Type                    | Syntax                                 | Example                          |
|-------------------------|----------------------------------------|----------------------------------|
| Static method           | <code>ClassName::method</code>         | <code>Math::abs</code>           |
| Instance method         | <code>obj::method</code>               | <code>System.out::println</code> |
| Constructor reference   | <code>ClassName::new</code>            | <code>Employee::new</code>       |
| Arbitrary object method | <code>ClassName::instanceMethod</code> | <code>String::toLowerCase</code> |

#### ◆ Example:

```
List<String> names = Arrays.asList("Kratos", "Akshay", "Java");
names.forEach(System.out::println); // method reference to println
```

## ✓ 7. JDBC (Java Database Connectivity) - Overview

JDBC = Java API to connect and interact with relational databases (e.g., MySQL, Oracle, PostgreSQL).

#### ◆ Steps:

#### 1. Load Driver:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

#### 1. Establish Connection:

```
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb", "user",  
"pass");
```

#### 1. Create Statement:

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users");
```

#### 1. Execute Query:

```
ResultSet rs = ps.executeQuery();
```

#### 1. Process Result:

```
while (rs.next()) {  
    System.out.println(rs.getString("name"));  
}
```

#### 1. Close Resources:

```
rs.close(); ps.close(); conn.close();
```

## 8. Web Server Installation + IDE Integration

If time permits:

### ◆ Apache Tomcat:

- Download from <https://tomcat.apache.org/>
- Extract & configure `CATALINA_HOME`
- Add Tomcat to Eclipse:

`Eclipse → Servers → New → Tomcat Server → Choose version and folder`

### ◆ Benefits:

- Run & debug JSP/Servlet apps directly from IDE
- Auto deploy WAR files
- View logs in console

## Summary

You've now touched:

- ITC with wait/notify vs sleep pitfalls
- volatile keyword for visibility
- User vs Daemon threads
- Random number generation
- Method reference types
- JDBC connection flow
- Web server setup intro

## ◆ Why **Executor** ?

The traditional way to create and manage threads ( `new Thread(r).start()` ) works, but:

- **Hard to manage** large number of threads
- **Difficult to reuse** threads (new one created every time)
- **No built-in pooling, scheduling, or control**

## ✓ Solution: **Executor Framework**

Introduced in Java 5 ( `java.util.concurrent` package)

## ◆ Core Interfaces and Classes

| Interface/Class                       | Description                                                     |
|---------------------------------------|-----------------------------------------------------------------|
| <code>Executor</code>                 | Base interface with <code>execute(Runnable)</code> method       |
| <code>ExecutorService</code>          | Extends Executor, adds features like submit(), shutdown(), etc. |
| <code>Executors</code>                | Factory class to create thread pools                            |
| <code>ThreadPoolExecutor</code>       | Concrete class with full control                                |
| <code>ScheduledExecutorService</code> | Executor with scheduling (like Timer replacement)               |

## ✓ Commonly Used Thread Pools

| Method                                           | Description                                     |
|--------------------------------------------------|-------------------------------------------------|
| <code>Executors.newFixedThreadPool(n)</code>     | Pool with fixed number of threads               |
| <code>Executors.newCachedThreadPool()</code>     | Creates new threads as needed, reuses idle ones |
| <code>Executors.newSingleThreadExecutor()</code> | One-thread executor                             |
| <code>Executors.newScheduledThreadPool(n)</code> | For scheduled, delayed, periodic tasks          |

## ✓ Example: **Fixed Thread Pool**

```
import java.util.concurrent.*;

public class ExecutorDemo {
    public static void main(String[] args) {
```

```

    ExecutorService executor = Executors.newFixedThreadPool(3);

    for (int i = 1; i <= 5; i++) {
        Runnable task = new MyTask(i);
        executor.execute(task);
    }

    executor.shutdown(); // No new tasks allowed
}

class MyTask implements Runnable {
    private int id;

    public MyTask(int id) {
        this.id = id;
    }

    public void run() {
        System.out.println("Task " + id + " executed by " + Thread.currentThread().getName());
        try { Thread.sleep(1000); } catch (InterruptedException e) {}
    }
}

```

🟢 Output will show **3 threads** running tasks concurrently and reusing them.

## ✅ Key ExecutorService Methods

| Method                          | Description                                                     |
|---------------------------------|-----------------------------------------------------------------|
| <code>submit(Callable)</code>   | Returns a <code>Future&lt;T&gt;</code> for getting result later |
| <code>invokeAll(list)</code>    | Runs all tasks, waits for all to complete                       |
| <code>shutdown()</code>         | Stops accepting new tasks; completes submitted                  |
| <code>shutdownNow()</code>      | Tries to stop all actively executing tasks                      |
| <code>awaitTermination()</code> | Waits for executor to terminate                                 |

## ✅ Callable + Future (Return Values + Exceptions)

```

import java.util.concurrent.*;

public class CallableDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        Callable<String> task = () -> {
            Thread.sleep(1000);
            return "Result from thread " + Thread.currentThread().getName();
        };
    }
}

```

```

};

Future<String> future = executor.submit(task);

System.out.println("Doing something else...");
System.out.println("Got: " + future.get()); // blocks until result is ready

executor.shutdown();
}
}

```

## ✓ ScheduledExecutorService

```

import java.util.concurrent.*;

public class SchedulerDemo {
    public static void main(String[] args) {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);

        Runnable task = () → System.out.println("Run at " + System.currentTimeMillis());

        scheduler.scheduleAtFixedRate(task, 2, 3, TimeUnit.SECONDS);
        // Initial delay: 2s, repeat every 3s
    }
}

```

## ✓ Custom ThreadPoolExecutor (Advanced Control)

```

ThreadPoolExecutor customPool = new ThreadPoolExecutor(
    2, 4, 60, TimeUnit.SECONDS,
    new LinkedBlockingQueue<>()
);

```

### Parameters:

- **corePoolSize**: Min number of threads kept alive
- **maximumPoolSize**: Max allowed threads
- **keepAliveTime**: Idle time before thread termination
- **workQueue**: Queue for waiting tasks

## ✓ Benefits Over Thread

- Thread pooling = **performance gain**
- **Cleaner code** using `Runnable` / `Callable`

- Exception handling, cancellation ( `Future.cancel` )
  - Task scheduling and concurrency support
- 

## ← END Summary

- `Executor` = Basic interface for executing tasks
  - `ExecutorService` = Advanced features: lifecycle, results, control
  - `Callable + Future` = Return values from tasks
  - `ScheduledExecutorService` = Scheduled/periodic task support
  - `ThreadPoolExecutor` = Custom configuration
- 

Would you like a **real-world use case**, like:

- Web server simulation with Executor
- Database operation using Callable
- Parallel file downloads

Or shall we go to **ReentrantLock**, **Atomic classes**, or **CompletableFuture**?